PR-JEPA: DECOUPLING LATENT PLANNING FROM FEW-STEP DISCRETE REALIZATION FOR NON-AUTOREGRESSIVE GENERATION OF CODE, LEGAL DOCUMENTS, AND CONTROL

Yeonseok Lee
SLING AI Inc.
Incheon, Republic of Korea
ylee@sling.ai.kr

ABSTRACT

Autoregressive (AR) decoders perform one forward pass per token, so sequential depth grows with output length and the key–value (KV) cache with context. Both costs limit closed-loop control and long structured outputs in which late content constrains early content: the imports and signatures of a program, the pragmas and array declarations of High-Level Synthesis (HLS) C code, the defined terms and cross-references of a contract. We propose **PR-JEPA** (Plan–Realize JEPA), which separates deciding *what* to generate from *emitting* it. A Joint-Embedding Predictive Architecture (JEPA) planner, trained with an exponential-moving-average target encoder and VICReg, samples a discrete mode and predicts a continuous plan Z_{target} over the output's structure, an abstract syntax tree or a clause outline. A cross-attention bridge conditions a few-step discrete flow-matching realizer that refines all positions in parallel in $K \leq 8$ steps, using a closed-form jump kernel tilted by plan-derived constraint energies and clipped to remain valid; a Logit-KL variant integrates a straight path in logit space instead. We prove that small VICReg penalties exclude rank-one collapse, that the K-step realizer amplifies plan errors by at most $(L_z/L_x)[(1 + L_x/K)^K - 1]$, that plan-mediated hard constraints survive fully parallel updates, that re-anchoring bounds latent error over any horizon, and how to set the uncertainty gate $H_t = \max(H_{\min}, H_{\max} e^{-\beta \bar{\sigma}_{t+1}})$ for a target risk. A roofline model predicts 9–21$\times$ lower single-request latency than AR decoding at $K = 8$, at about $1/K$ of AR's saturated throughput. We pre-register evaluation on general-purpose code, HLS-C, contract drafting, and robot manipulation; end-to-end measurements are pending.

Keywords Joint-embedding predictive architecture · Discrete flow matching · Non-autoregressive generation · Code generation · High-level synthesis · Legal document drafting · Robot control

1 Introduction

Real-time controllers and long structured programs share one requirement that autoregressive decoders cannot meet cheaply: a globally consistent output, produced with a sequential depth that does not grow with its length.

1.1 The sequential bottleneck of autoregression

An AR model factorizes the output distribution left to right,

$$p_\theta(w \mid c) = \prod_{i=1}^{N} p_\theta\left(w^i \mid w^{<i}, c\right), \tag{1}$$

so sampling N tokens requires N dependent forward passes: the sequential depth is $\Theta(N)$. At small batch, each decode step reads every weight from memory, so latency per token is set by memory bandwidth rather than arithmetic [1]. Each generated token also leaves keys and values in the cache. For a model with L layers, h_{kv} key–value heads of width d_h,

and b bytes per element, the cache costs

$$M_{\mathrm{KV}}(T) = 2\,L\,h_{kv}\,d_h\,b \cdot T \text{ bytes for a context of } T \text{ tokens.} \tag{2}$$

For the public Qwen2.5-Coder-7B configuration (28 layers, 4 KV heads of width 128, bf16) this is 56 KiB per token; a multi-head 1.3B model with 16 KV heads at the same width costs 192 KiB per token. Long outputs at high batch therefore become memory-bound, a constraint that paged allocation mitigates but does not remove [2].

Two further costs are structural. First, a token, once emitted, conditions everything after it, so an early error cannot be revised and compounds. Second, in embodied control, a vision-language-action (VLA) model that tokenizes an H-step chunk of a d_a-dimensional action with q tokens per component needs $N = q\,H\,d_a$ sequential passes per chunk; inference rates of about 1–5 Hz have been reported for RT-2 [3], as summarized by Lee [4].

Structured outputs expose the second cost sharply, because what comes late determines what must come early. In a general-purpose program, the imports, signatures and type annotations near the top must match calls and uses written further down. In HLS-C, a pipelining directive in an inner loop reaches its initiation interval only if the arrays it reads are partitioned to supply enough ports per cycle, yet the partition directive and the array's bit-width are declared near the top of the function. In a contract, every capitalized term must match a definition, usually collected in an opening definitions section, and every cross-reference such as "Section 9.2" must point to a clause whose number depends on everything drafted before it. A left-to-right decoder must commit to the early part before it has decided the later part that determines it.

We stress what this argument does *not* claim. AR models are strong in all three domains. Fine-tuned with abstract-syntax-tree (AST) guidance, they reach 92.95% Synth@1 and 57.69% functional Pass@1 on a VerilogEval-derived HLS benchmark [5], and the instruction-tuned Qwen2.5-Coder reaches 90.2% HumanEval Pass@1 as reported by Gong et al. [6]. The case for parallel realization is latency inside interactive loops (design-space exploration, code completion, contract negotiation) and the global consistency of decisions that are fixed early but justified late. For legal text, reliability is the binding constraint: commercial AI legal research tools still hallucinate in 17–33% of answers [7]. PR-JEPA does not establish factual or legal correctness, but it makes the internal consistency of a document a checkable property of its plan.

1.2 The dilemma of discrete parallel generation

Generating all positions at once removes the sequential depth but introduces a different error. A one-shot non-autoregressive (NAR) decoder outputs a product of per-position distributions, and the best such product approximates the true conditional only up to its total correlation (Proposition 1, Section 4). When the target is multimodal, the product of marginals places mass on *mixtures* of modes. Suppose two valid HLS designs exist for one specification: partition factor 4 with initiation interval (II) 1, or partition factor 2 with II 2. Independent per-position sampling produces the infeasible pair (factor 2, II 1) a quarter of the time. A confidentiality agreement sampled the same way blends a mutual and a one-way template, binding "each party" in one clause and only "the Recipient" in the next.

Iterative refinement repairs this by re-sampling positions conditioned on a partially generated draft. Discrete diffusion [8, 9, 10] and discrete flow matching (DFM) [11, 12] do so with hundreds to thousands of steps, because each step updates positions independently given the current state and so incurs a factorization error of its own. Few-step methods attack the step count directly: FS-DFM trains for an explicit step budget and reaches the perplexity of a 1,024-step DFM baseline in 8 steps [13]. The alternative of diffusing continuous token embeddings and rounding at the end discards categorical structure, which is why Logit-KL flow matching instead moves along KL geodesics of the probability simplex, i.e., straight lines in logit space [14]. Each of these realizers must still *discover* global structure from token-level noise, using coordinate-wise updates.

1.3 Plan in latents, realize in parallel

We separate the two jobs. A **global latent planner** decides the structure of the output once: it selects a discrete mode u, which resolves multimodality, and predicts a continuous plan $Z_{\text{target}} \in \mathbb{R}^{M \times d}$ over the output's structural slots (the regions of an abstract syntax tree for code, the clause outline for a contract) with a JEPA predictor whose representation space is kept full-rank by VICReg and an EMA target encoder. A **parallel realizer** then transports a simple source distribution to the token sequence in K steps, conditioned on Z_{target} through cross-attention. The realizer is either a continuous-time Markov chain (CTMC) with a closed-form few-step jump kernel, tilted by constraint energies derived from the plan and clipped to stay a valid kernel, or a deterministic ODE in logit space. One architecture serves general-purpose code, HLS-C and legal documents; only the structural view, the mode vocabulary and the energies change (Table 1). For continuous control the same bridge feeds a rectified-flow action solver, as in the Vision-Predictor-Action

(VPA) framework of our earlier work [4]. Including the planner, a decision costs $K + 3$ sequential network evaluations, independent of N.

1.4 Contributions

1. **Architecture.** PR-JEPA: a mode selector and JEPA planner, a cross-attention bridge whose keys and values are computed once per generation, a conditional FS-DFM realizer with an energy-tilted, clipped jump kernel, and a Logit-KL alternative. One architecture is instantiated for general-purpose code, HLS-C and legal documents by changing only the structural view, the mode vocabulary and the plan-mediated constraints (Section 3, Table 1).
2. **Theory.** (a) An identity that equates the error of one factorized parallel step with conditional total correlation and explains why the plan must carry a *mode* variable, not only a regression target. (b) A Gershgorin bound under which VICReg penalties imply a positive-definite covariance, a stable-rank bound when they do not vanish, and an exact gradient norm for the variance hinge. (c) A sensitivity bound for K Euler steps of the realizer, token-invariance margins, and a coupling bound for the CTMC realizer that grows only as the harmonic number H_K. (d) Proof that plan-mediated hard constraints, such as declared identifiers, bit-widths, defined terms and cross-references, survive fully parallel updates. (e) Horizon-independent error containment under sub-goal re-anchoring, which recovers the VPA transition bound $(\tau+\delta_\eta)/\mu_E$ under explicit assumptions. (f) A rule for choosing β in the uncertainty gate that guarantees a target risk level (Section 4).
3. **Systems analysis.** A roofline model that predicts where NAR realization wins (single-request latency, 9–21$\times$ at $K = 8$) and where it loses (saturated throughput, about $1/K$ of AR) (Section 5.4).
4. **Pre-registered evaluation.** Tasks, baselines, metrics, hypotheses, and ablations for HLS-C generation, general-purpose code in Python and other languages, contract drafting from term sheets, and LIBERO manipulation, with every number not yet measured marked as such (Section 5). Each theorem was also checked numerically on random instances; no bound was violated.

2 Related Work

PR-JEPA combines three lines of work that have so far been studied apart: JEPA world models, few-step discrete flows, and structure-aware generation of programs, hardware designs, and legal documents.

2.1 Self-supervised world models and JEPA

I-JEPA predicts the representations of masked target blocks from a context block, with targets produced by an EMA copy of the encoder [15]; V-JEPA extends feature prediction to video [16], and V-JEPA 2 post-trains an action-conditioned predictor for planning in representation space [17]. Collapse is prevented either by architectural asymmetry with an EMA target and stop-gradient, as in BYOL [18], or by explicit variance and covariance penalties, as in VICReg [19]; we use both.

D-JEPA [20] is the closest generative use of JEPA, and it differs from our setting in three ways. It reinterprets JEPA as generalized next-token prediction and therefore generates *autoregressively*; it models continuous image tokens with per-token diffusion or flow-matching losses rather than discrete tokens; and it does not use VICReg. Its reported FIDs (1.58 for D-JEPA-L and 1.54 for D-JEPA-H with classifier-free guidance on ImageNet 256$\times$256) concern a different modality, and we do not use them as evidence for discrete generation.

Flow-JEPA [21] is a different method despite the similar name. It places flow matching *inside* the JEPA world model: future latent states are generated from Gaussian noise, conditioned on the current observation and actions, so that stochastic trajectory-level prediction replaces pointwise transition regression; the authors report higher planning success than their baselines, particularly under localized visual noise. PR-JEPA keeps a regression predictor with a discrete mode and uses flow matching *after* the planner, to turn the plan into discrete tokens or actions. The two are complementary: a stochastic latent predictor of that kind is one way to obtain the sampled, mode-resolving plan that Proposition 1 requires, and could replace our mode selector where modes are hard to enumerate.

For control, our earlier Vision-Predictor-Action (VPA) framework replaces the autoregressive decoder of a VLA with a JEPA latent predictor, a neuro-symbolic primitive selector, and a flow-matching action solver [4]. Its sequential depth is $K + 3$ evaluations per decision, independent of the horizon; its reference implementation takes 10.5–14.2 ms per decision step on an Apple M4 for $K \in \{1, 2, 3\}$ and $H \leq 64$, and reaches 40.0% success on LIBERO-10 with two cameras over 200 episodes (95% Wilson interval 33.5–46.9%) from a single training run. VPA presents its Lipschitz error-containment argument as an expectation to be verified, not a guarantee; we keep that stance and make

the assumptions explicit (Section 4.4). Concurrent work conditions a flow-matching action expert on a future-latent predictor's key–value states [22] and uses an action-conditioned JEPA across the VLA lifecycle [23]. PR-JEPA shares the bridge design for control; its new elements are a discrete-token realizer, plan-mediated constraints, and the analysis.

2.2 Continuous and discrete flow matching

Flow matching learns a velocity field whose ODE transports a source distribution to the data [24]; rectified flow chooses straight interpolation paths and straightens them further by reflow, so that one or two Euler steps suffice [25]. Robot policies use these ideas for action chunks [26, 27], and reflow regularization of flow policy gradients has been used to obtain one-step actions [28].

For discrete data, D3PM [8], score-entropy diffusion [9], and masked diffusion [10, 29] define corruption processes on tokens. Campbell et al. [11] formulate discrete flows as CTMCs; Discrete Flow Matching (DFM) builds probability paths from schedulers and reaches 6.7% Pass@1 on HumanEval at 1.7B parameters [12]. FS-DFM conditions the model on a step budget, uses a closed-form cumulative scalar update so that large steps stay on the simplex, and distills from long-run teachers; with 8 steps it matches the perplexity of a 1,024-step DFM baseline for 1,024-token generation, up to 128× faster sampling [13]. Logit-KL flow matching interpolates along KL geodesics, which are linear in logit space, and proves that maximizing the conditional likelihood $p_\theta(x_1 \mid x_t)$ recovers the flow-matching velocity [14]; on MBPP code infilling with 10% of lines removed it raises Pass@1 from 11.1 to 17.4 and Pass@10 from 25.5 to 29.2 over DFM. Discrete Guidance tilts CTMC rate matrices by likelihood ratios to sample from conditional distributions [30]; Section 3.3 shows that our energy barrier is a clipped instance of it. Earlier NAR translation [31, 32] identified the multimodality problem that Proposition 1 formalizes.

AR inference can also be accelerated without changing its semantics, for example by speculative decoding [33]; such methods reduce latency but keep a left-to-right commitment order.

2.3 Structure-aware generation of code, hardware, and legal documents

General-purpose code. HumanEval [34] and MBPP [35] score Python programs by unit tests, and MultiPL-E translates both into 18 further programming languages [36]. Non-autoregressive code models have advanced quickly. DFM reached 6.7% HumanEval Pass@1 at 1.7B parameters [12]. The 7B masked-diffusion model DiffuCoder reaches 67.1% HumanEval and 74.2% MBPP Pass@1 as a base model, against 61.6% and 75.9% for the AR Qwen2.5-Coder base it is compared with; after instruction tuning the AR model leads clearly, 90.2% against 72.0% on HumanEval [6]. These models refine tokens without an explicit plan; PR-JEPA adds one, built from the program's AST.

Hardware code. VerilogEval established functional evaluation of LLM-generated hardware [37]. SAGE-HLS fine-tunes Qwen2.5-Coder-7B on about 16.7K HLS-C programs ported from verified Verilog, with AST-guided instruction prompts, and evaluates synthesis with AMD Vitis HLS and functional correctness with constrained RTL simulation [5]. Its reported Synth@1 and Pass@1 are 92.95% and 57.69%, against 52.56% and 22.44% for the base model; the frequently quoted 75% is Pass@10 (75.64%). PRISM injects AST, control-flow, and data-flow graphs into a frozen code LLM through gated cross-attention for pragma optimization [38]. These systems condition an AR decoder on structure *extracted* from given code; PR-JEPA predicts the structure of code that does not yet exist and conditions a parallel realizer on it.

Legal documents. Public contract datasets are built for understanding rather than drafting. CUAD contains 510 contracts from SEC EDGAR filings with 13,101 expert-labeled clauses in 41 categories [39]. LEDGAR labels almost 100,000 provisions from over 60,000 contracts with more than 12,000 labels [40]. ContractNLI asks whether each of a fixed set of hypotheses is entailed by, contradicted by, or not mentioned in each of 607 contracts, with evidence spans [41]. Reliability remains the central concern: commercial AI legal research tools hallucinated in 17–33% of answers in a systematic evaluation [7]. We use these resources to build plans (clause outlines, defined terms, cross-references) and to score generated contracts, not as generation benchmarks in their own right.

3 The PR-JEPA Architecture

PR-JEPA computes a plan once and then realizes all N output positions in K parallel steps conditioned on it; Figure 1 shows the data flow and Algorithm 1 the sampler.

Notation. The vocabulary is $\mathcal{V}$ with $|\mathcal{V}| = S$; an output is $w = (w^1, \ldots, w^N) \in \mathcal{V}^N$; the context c is a specification with a function signature (general-purpose code and HLS-C), a term sheet (contracts), or an observation with an

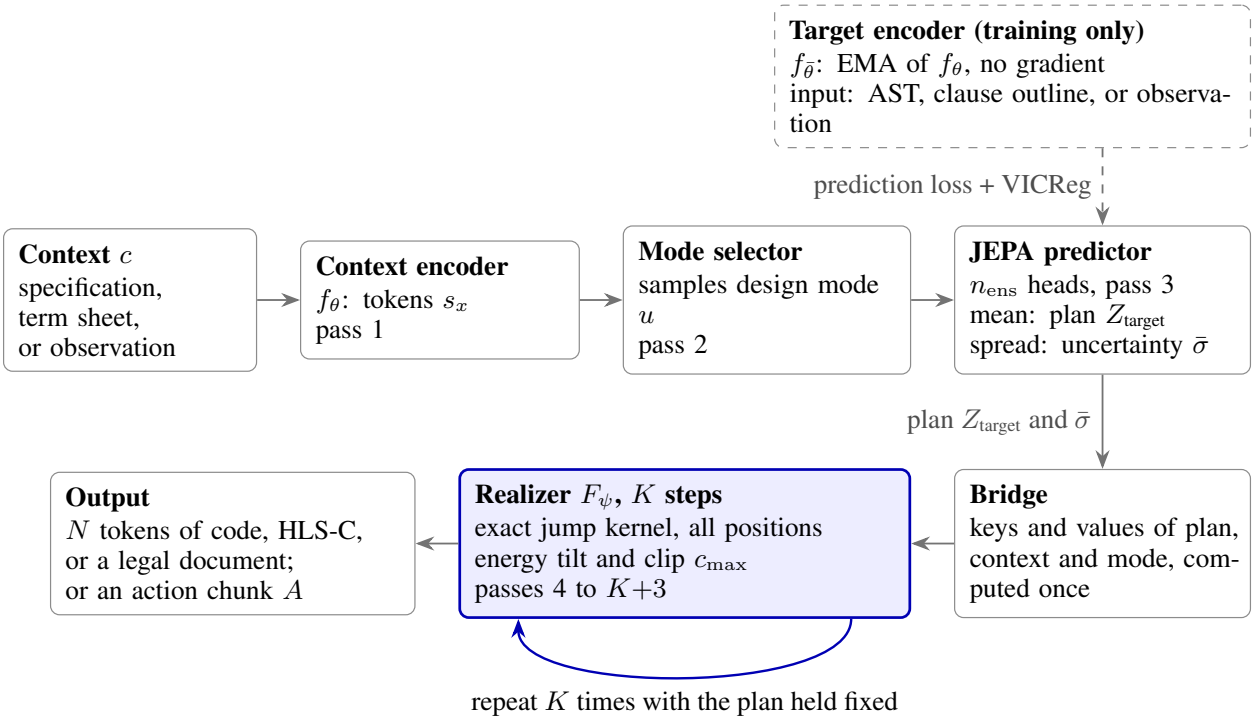


Figure 1: The planner runs three passes once per output or decision; the realizer then makes K parallel passes over all positions with the plan held fixed, so the sequential depth is $K + 3$ whatever the output length N. The dashed target branch exists only during training, and $\bar{\sigma}$ also sets the executed or committed horizon (Section 4.5).

instruction (control). The plan is always written $Z_{\text{target}} \in \mathbb{R}^{M\times d}$. In the VICReg terms, $Z \in \mathbb{R}^{n\times d}$ denotes a batch of n embeddings with columns z^j.

3.1 Latent planner

The planner has four parts. A context encoder f_θ maps c to tokens $s_x \in \mathbb{R}^{M_c\times d}$. A target encoder $f_{\bar{\theta}}$ maps a structured view y of the training target to $s_y \in \mathbb{R}^{M\times d}$, one embedding per structural slot: regions of the abstract syntax tree (AST) for code, the clause outline for a contract, and the future observation for control, as in VPA (Table 1). A mode selector $\pi^h_\phi(u \mid s_x)$ chooses a discrete mode $u \in \mathcal{U}$ from a domain vocabulary, for example a directive configuration for HLS-C, an algorithm or library choice for a program, a template variant and governing law for a contract, or VPA's primitive token for control. Finally, a predictor g_ϕ produces the plan,

$$Z_{\text{target}} = \hat{s}_y = g_\phi\,(s_x,\, u,\, \Delta) \in \mathbb{R}^{M\times d}, \tag{3}$$

where Δ are slot queries (AST positions or temporal offsets).

The mode is not decoration. A predictor trained with squared error estimates the conditional *mean* of s_y; when a specification admits several designs, that mean lies between their embeddings and need not decode to any valid program. Sampling u first makes the remaining target close to unimodal, which is what a single regression can represent (Proposition 1).

Target encoder. The target parameters receive no gradient and follow an exponential moving average whose momentum rises linearly to one [15]:

$$\bar{\theta} \leftarrow m_k\,\bar{\theta} + (1 - m_k)\,\theta, \qquad m_k = m_0 + (1 - m_0)\,\frac{k}{T_{\text{train}}}, \qquad m_0 = 0.996. \tag{4}$$

Table 1: Domain instantiations: what the plan encodes, what the mode chooses, and which constraints the plan enforces.

Domain	Structural view y (plan slots)	Mode u	Plan-mediated constraints (energies)	Output and realizer
General-purpose code (Python, C++, Java, Rust)	AST regions from a parser such as tree-sitter: imports, signatures, class skeletons, declared identifiers with types	Algorithm or data-structure choice; libraries used	Only identifiers the plan declares may be referenced; argument counts match planned signatures; imports match planned dependencies	Programs up to a few thousand tokens; CTMC or Logit-KL
HLS-C	AST regions: interface, array declarations with directives, loop nests with directives	Directive classes: pipeline or not and an II class per loop, a partition scheme per array	Bit-widths from the plan; directive legality (a partition factor divides the planned dimension, II at least 1)	Kernels up to a few thousand tokens; CTMC or Logit-KL
Legal documents (contracts)	Clause outline with section numbers; defined-term inventory; party, date and amount slots; cross-reference graph	Template variant (for example mutual or one-way confidentiality), governing law, dispute forum, liability-cap style	Capitalized terms only from the plan's inventory; cross-references resolve to planned sections; party names, dates and amounts copied from plan slots	Often several thousand tokens; CTMC, section by section with re-anchoring (Theorem 3)
Robot control (VPA)	Future observation latent	Primitive skill token	None needed; actions are continuous	An $H \times d_a$ action chunk; rectified-flow ODE

Objective. With sg the stop-gradient operator, B the batch size, Z_x and $\hat{Z}$ the context and predicted embeddings of the batch stacked into $n = BM$ rows, and $\mathcal{L}_{\text{CE}}$ the selector's cross-entropy against modes extracted from the reference,

$$\begin{aligned}\mathcal{L}_{\text{plan}} = \frac{1}{BM}\sum_{b=1}^{B}\left\|\hat{s}_y^{(b)} - \text{sg}\left(s_y^{(b)}\right)\right\|_F^2 \\ + \lambda_v\left[v(Z_x) + v(\hat{Z})\right] + \lambda_c\left[c(Z_x) + c(\hat{Z})\right] + \lambda_u\,\mathcal{L}_{\text{CE}}\left(\pi_\phi^h\right).\end{aligned} \tag{5}$$

The prediction term plays VICReg's invariance role. The regularizers act on the two branches that receive gradients; applied to the stop-gradient target branch they would have no effect. The variance hinge uses an ϵ-regularized standard deviation,

$$\begin{aligned}v(Z) = \frac{1}{d}\sum_{j=1}^{d}\max\left(0,\ \gamma - \sqrt{\text{Var}(z^j) + \epsilon}\right), \\ \text{Var}(z^j) = \frac{1}{n-1}\sum_{i=1}^{n}\left(z_i^j - \bar{z}^j\right)^2,\end{aligned} \tag{6}$$

and the covariance penalty is the scaled squared off-diagonal mass of the sample covariance,

$$C(Z) = \frac{1}{n-1}\sum_{i=1}^{n}\left(z_i - \bar{z}\right)\left(z_i - \bar{z}\right)^\top, \qquad c(Z) = \frac{1}{d}\sum_{j\neq k}\left[C(Z)\right]_{jk}^2. \tag{7}$$

We start from VICReg's published weights, $25:25:1$ for invariance, variance, and covariance, with $\gamma = 1$ [19], and set $\epsilon = 10^{-4}$; Proposition 3 explains why ϵ must stay well below γ^2.

Epistemic uncertainty. The predictor carries n_{ens} lightweight heads on a shared trunk, in the manner of deep ensembles [42]. The plan is their mean and the uncertainty their spread:

$$\begin{aligned} Z_{\text{target}} &= \frac{1}{n_{\text{ens}}} \sum_{e=1}^{n_{\text{ens}}} \hat{s}_y^{(e)}, \\ \bar{\sigma} &= \left(\frac{1}{n_{\text{ens}} M d} \sum_{e=1}^{n_{\text{ens}}} \left\| \hat{s}_y^{(e)} - Z_{\text{target}} \right\|_F^2 \right)^{1/2}. \end{aligned} \tag{8}$$

Section 4.5 uses $\bar{\sigma}$ to set the execution or commitment horizon, after calibrating it on held-out rollouts.

3.2 Bridge: latent conditioning by cross-attention

The realizer's conditional velocity is

$$v_t\left(w_t \mid Z_{\text{target}}\right) = F_\psi\left(w_t,\, t,\, h,\, Z_{\text{target}}\right), \tag{9}$$

a bidirectional transformer with L_r blocks; we write F_ψ rather than f_θ to keep it distinct from the context encoder. Each block applies self-attention over the N positions, cross-attention to the conditioning tokens $\tilde{Z} = [Z_{\text{target}};\, s_x;\, e(u)]/\gamma$, and an MLP, with adaptive layer-norm modulation by embeddings of the time t and the step size h (FS-DFM's step-budget conditioning). Scaling by the constant $1/\gamma$ instead of normalizing $\tilde{Z}$ keeps the bridge Lipschitz in the plan; a layer norm's Lipschitz constant grows as a token's feature spread shrinks. Block ℓ computes

$$\mathrm{CA}_\ell(H, \tilde{Z}) = \mathrm{softmax}\left(\frac{(HW_Q^\ell)(\tilde{Z}W_K^\ell)^\top}{\sqrt{d_k}} \right) \tilde{Z}W_V^\ell. \tag{10}$$

Because $\tilde{Z}$ does not change during sampling, the keys $\tilde{Z}W_K^\ell$ and values $\tilde{Z}W_V^\ell$ are computed once per generation and reused at all K steps. Their memory is $2L_r\tilde{M}d_k b$ bytes, independent of N: 48 MiB for $\tilde{M} = 256$ conditioning tokens, $L_r = 24$, $d_k = 2048$ and bf16, against 288 MiB of AR KV cache for a 1,024-token output of a multi-head 1.3B model with a 512-token prompt. Spectral normalization of W_K^ℓ and W_V^ℓ [43] bounds the bridge's sensitivity to $\tilde{Z}$ for bounded queries. Dot-product self-attention is not globally Lipschitz [44], so the regularity assumption of Section 4 is stated on a bounded domain. During training the conditioning is replaced by a learned null token with probability 0.1, which enables predictor-free guidance on rates [30] and robustness to a missing plan.

3.3 Conditional FS-DFM realizer

Probability path. The source p_0 is uniform over $\mathcal{V}$ (as in the released FS-DFM checkpoints) or a mask token. Each position follows the mixture path

$$\begin{gathered} p_t\left(w^i \mid w_0^i, w_1^i\right) = (1-\kappa_t)\,\delta_{w_0^i}\left(w^i\right) + \kappa_t\,\delta_{w_1^i}\left(w^i\right), \\ \kappa_0 = 0,\ \kappa_1 = 1,\ \dot{\kappa}_t \geq 0. \end{gathered} \tag{11}$$

The denoiser is trained by cross-entropy, conditioned on the frozen plan:

$$\mathcal{L}_{\text{real}}(\psi) = \mathbb{E}_{t,\,(c,w_1),\,w_0,\,w_t}\left[-\sum_{i=1}^{N} \log p_{1|t}^{\psi}\left(w_1^i \mid w_t,\, t,\, h,\, \mathrm{sg}(Z_{\text{target}})\right) \right]. \tag{12}$$

Generator. The rate at which position i moves to token j is

$$u_t^i(j \mid w_t, Z) = \frac{\dot{\kappa}_t}{1-\kappa_t}\left[p_{1|t}^{\psi}(j \mid w_t, Z) - \delta_{w_t^i}(j) \right], \tag{13}$$

which defines a CTMC whose generator changes one position at a time and whose marginals obey the Kolmogorov forward equation [11, 12]:

$$\partial_t\, p_t(w) = \sum_{w'} p_t(w')\, Q_t(w', w), \qquad Q_t\left(w,\ w^{i \leftarrow j}\right) = u_t^i(j \mid w, Z) \quad (j \neq w^i). \tag{14}$$

Why not Euler. The Euler kernel $\delta_{w_t^i} + h\,u_t^i$ is a probability distribution only if $h\dot{\kappa}_t/(1-\kappa_t) \le 1$. The linear schedule meets this at the grid points $t_k = k/K$, with equality at the last step; the schedule $\kappa_t = \sqrt{t}$ violates it without bound as $t \to 0$. Few-step sampling needs a kernel that is valid for any step.

Exact few-step kernel. Holding the posterior fixed over $[t, t+h]$, the mixture path gives the closed-form jump mass and kernel

$$\begin{aligned} \pi_{t,h} &= \frac{\kappa_{t+h} - \kappa_t}{1-\kappa_t} \in [0,1], \\ P\big(w_{t+h}^i = j \mid w_t, Z\big) &= (1-\pi_{t,h})\,\delta_{w_t^i}(j) + \pi_{t,h}\,p_{1|t}^{\psi}(j \mid w_t, Z). \end{aligned} \tag{15}$$

Lemma 1 (exactness)**.** *Conditional on (w_0, w_1) and with the oracle posterior $\delta_{w_1^i}$, this kernel maps $p_t(\cdot \mid w_0, w_1)$ to $p_{t+h}(\cdot \mid w_0, w_1)$ for every h.*

Proof. A position already at w_1^i (probability κ_t) stays there, since a jump also lands on w_1^i; otherwise it moves with probability $\pi_{t,h}$. The arrival probability is $\kappa_t + (1-\kappa_t)\pi_{t,h} = \kappa_{t+h}$. □

With a learned posterior, the only errors left are the posterior's own and the factorization across positions, which Proposition 1 quantifies. FS-DFM's cumulative scalar update rests on the same observation [13]; we do not claim the two update rules coincide in every detail.

Energy barrier and safety clipping. Let $E^i(j \mid w_t^{-i}, Z) \ge 0$ be a constraint energy, zero for admissible tokens, and $T > 0$ a temperature. The barrier tilts and clips the jump rates,

$$\begin{aligned} &P\big(w_{t+h}^i = j \mid w_t^i = a,\, w_t,\, Z\big) \\ &\quad \propto \mathrm{clip}\Big(u_t^i(j \mid w_t, Z)\, e^{-E^i(j|w_t^{-i}, Z)/T},\ 0,\ u_{\max}\Big), \qquad j \ne a, \end{aligned} \tag{16}$$

and we normalize it with the exact jump mass:

$$\begin{aligned} \tilde{\rho}^{\,i}(j) &= \min\Big(p_{1|t}^{\psi}(j \mid w_t, Z)\, e^{-E^i(j|w_t^{-i}, Z)/T},\ c_{\max}\Big), \\ \tilde{K}^i(j) &= \begin{cases} \pi_{t,h}\,\tilde{\rho}^{\,i}(j), & j \ne a, \\ 1 - \pi_{t,h}\sum_{j' \ne a}\tilde{\rho}^{\,i}(j'), & j = a. \end{cases} \end{aligned} \tag{17}$$

For $j \ne a$ the rate is $u_t^i(j) = \frac{\dot{\kappa}_t}{1-\kappa_t}\,p_{1|t}^{\psi}(j)$, so the clipped term above equals the tilted rate up to the positive factor $\dot{\kappa}_t/(1-\kappa_t)$, which the exact mass $\pi_{t,h}$ replaces, with $c_{\max} = u_{\max}(1-\kappa_t)/\dot{\kappa}_t$. Rejected jump mass becomes probability of staying. The lower clip at 0 matters when rates come from extrapolating guidance, $(1+\omega)\,u(\cdot \mid Z) - \omega\,u(\cdot \mid \varnothing)$, which can turn negative. Proposition 4 shows $\tilde{K}^i$ is a valid kernel for every $T > 0$ and $c_{\max} \in (0, 1]$.

The tilt is a clipped instance of Discrete Guidance. If an admissibility predictor satisfies $p(\text{admissible} \mid w) \propto e^{-E(w)/T}$, exact predictor guidance multiplies each rate by $e^{-[E(j)-E(a)]/T}$ [30]. Our tilt equals this whenever the current token is admissible ($E(a) = 0$) and is more conservative otherwise. A repair-aware variant uses the difference $E(j) - E(a)$; its multiplier can exceed one, which is precisely when the upper clip $c_{\max}$ is required to keep the kernel valid.

Energy terms for HLS-C. We use $E = \sum_k \lambda_k E_k$ with four terms:

1. *Grammar:* $E_{\text{gram}} = \infty$ for tokens that no completion of the current draft can parse at that position, from an incremental parser over the draft. Over-approximation is safe: the check only needs never to reject a valid token.
2. *Bit-width:* the plan's slot decoder predicts a width $\hat{b}_v$ for each variable v; a token declaring `ap_uint<b>` or `ap_int<b>` for v costs $E_{\text{bw}} = \Big(\log_2(b/\hat{b}_v)\Big)^2$.
3. *Directive legality:* for example, an `ARRAY_PARTITION` factor must divide the partitioned dimension recorded in the plan, and a `PIPELINE` II must be at least 1.
4. *Checkers (optional):* static analyzers run on the draft between steps, such as a memory-port conflict checker, contribute further terms.

Terms 2 and 3 depend on (j, Z) only. Proposition 4 shows that such plan-mediated hard constraints survive fully parallel updates, whereas draft-dependent terms 1 and 4 are guaranteed only under sequential updates or a final repair pass. This is a design rule: express every constraint that can be expressed through the plan.

Algorithm 1 Few-step sampling with PR-JEPA. Each step is one forward pass of F_ψ over all positions in parallel.

Require: context c; step budget K; schedule κ; temperature T; clip $c_{\max}$; energy E

1: $s_x \leftarrow f_\theta(c)$ ▷ context encoder (pass 1)
2: $u \sim \pi_\phi^h(\cdot \mid s_x)$ ▷ mode selector (pass 2)
3: $\{\hat{s}^{(e)}\}_{e=1}^{n_{\text{ens}}} \leftarrow g_\phi^{(e)}(s_x, u, \Delta)$ ▷ predictor heads (pass 3)
4: $Z \leftarrow \frac{1}{n_{\text{ens}}} \sum_e \hat{s}^{(e)}$; $\bar{\sigma} \leftarrow$ ensemble spread
5: $(K_\ell, V_\ell)_\ell \leftarrow$ bridge projections of $[Z;\, s_x;\, e(u)]/\gamma$ ▷ once; fused into pass 4
6: $w \leftarrow N$ tokens drawn from p_0 ▷ uniform or mask source
7: **for** $k = 0, \dots, K-1$ **do** ▷ passes 4 to $K+3$
8: $t \leftarrow k/K$; $h \leftarrow 1/K$; $\pi \leftarrow (\kappa_{t+h} - \kappa_t)/(1-\kappa_t)$
9: $P \leftarrow p^\psi_{1|t}(\cdot \mid w, t, h, Z)$ ▷ $N \times S$ posterior, one forward pass
10: **for all** positions i in parallel **do**
11: $\rho_i(j) \leftarrow \min\big(P_i(j)\, e^{-E_i(j|w,Z)/T},\, c_{\max}\big)$ for $j \neq w_i$
12: with probability $\pi \sum_j \rho_i(j)$: draw $j \sim \rho_i / \sum_{j'} \rho_i(j')$ and set $w_i \leftarrow j$
13: **end for**
14: **end for**
15: **return** w

Energy terms for the other domains. For general-purpose code, a token that references an identifier absent from the plan's declared set costs $E = \infty$, or a finite penalty when the planner may have omitted a helper; argument counts are checked against the planned signatures, and the grammar term is that of the target language. For contracts, a capitalized term outside the plan's defined-term inventory, a cross-reference to a section the outline does not contain, and a party name, date or amount that differs from its plan slot each cost $E = \infty$; drafting conventions enter as soft terms. All of these depend on (j, Z) only, so Proposition 4(d) covers them.

Lines 1–3 and the K realizer passes give a sequential depth of $K + 3$ network evaluations, the same count as a VPA decision step [4], and independent of N. If the mode slots are strongly coupled, a small autoregressive head over the M_u mode slots replaces line 2 and adds M_u short passes. With the linear schedule the final step has $\pi = 1$, so every position is redrawn once from the tilted posterior.

3.4 Logit-KL realizer

The alternative realizer moves a continuous state through logit space. One-hot targets have infinite logits, so we use smoothed targets $x_1^\varepsilon = (1-\varepsilon)\, e_k + (\varepsilon/S)\,\mathbb{1}$, whose logits are $l_1 = a\, e_k + b\mathbb{1}$ with

$$a = \log \frac{1 - \varepsilon + \varepsilon/S}{\varepsilon/S}, \qquad b = \log \frac{\varepsilon}{S}. \tag{18}$$

From a source $l_0 \sim \mathcal{N}(0, s^2 I)$ per position, the path and its projection to the simplex are

$$l_t = (1-t)\, l_0 + t\, l_1, \qquad x_t = \mathrm{softmax}(l_t) \propto x_0^{1-t} \odot x_1^t, \tag{19}$$

the KL (exponential-family) geodesic [14]. Along a conditional path, $\dot{l}_t = l_1 - l_0 = (l_1 - l_t)/(1-t)$.

Lemma 2 (posterior parameterization)**.** *The marginal velocity is affine in the posterior over the target token:*

$$\begin{aligned} v^\star(l,t) = \mathbb{E}\left[l_1 - l_0 \mid l_t = l\right] &= \frac{a\, p(\cdot \mid l_t = l) + b\mathbb{1} - l}{1-t}, \\ p(k \mid l_t = l) &= P\left(w_1 = k \mid l_t = l\right). \end{aligned} \tag{20}$$

Hence a network trained with cross-entropy to output $p_\psi(k \mid l_t, t, Z)$ yields $v^\star$ exactly at the population optimum, and the term $b\mathbb{1}$ can be dropped because softmax ignores constant shifts.

Proof. From $l_t = (1-t)l_0 + t\, l_1$, $l_1 - l_0 = (l_1 - l_t)/(1-t)$. Taking the conditional expectation given $l_t = l$ and using $\mathbb{E}[l_1 \mid l_t] = a\, \mathbb{E}[e_{w_1} \mid l_t] + b\mathbb{1} = a\, p(\cdot \mid l_t) + b\mathbb{1}$ gives the claim. □

This specializes the likelihood–velocity result of Sevriugov et al. [14] to smoothed targets; for sequences, the posterior of position i conditions on the whole state l_t and on Z.

Sampler. Euler integration on $t_k = k/K$ gives

$$l_{k+1} = l_k + \frac{1}{K} \cdot \frac{a\, p_\psi(\cdot \mid l_k, t_k, Z) - l_k}{1 - t_k}. \tag{21}$$

At the last step $t_{K-1} = 1 - 1/K$, so the update returns $l_K = a\, p_\psi(\cdot \mid l_{K-1}, t_{K-1}, Z)$: the singularity at $t = 1$ is never evaluated, and the final logits are the scaled posterior. Tokens are decoded as $\hat{w}^i = \arg\max_j l_K^i(j)$, and energies enter by tilting the posterior, $\tilde{p} \propto p\, e^{-E/T}$.

Trade-offs. The Logit-KL realizer is deterministic given l_0 and Lipschitz-analyzable (Theorem 2 applies with the state $x = l$). Its state, however, is $N \times S$ numbers rather than N integers: about 0.3 GB in bf16 for $N = 1{,}024$ and the 152,064-token Qwen2.5 vocabulary. For continuous control, the realizer is a rectified-flow ODE over the action chunk $A \in \mathbb{R}^{H \times d_a}$ with the same bridge, which is VPA's solver, and Theorem 2 applies to it directly. Contracts, with natural-language vocabularies and outputs of several thousand tokens, default to the CTMC realizer generated section by section.

4 Theoretical Analysis

The results below say when a plan helps a parallel step, when VICReg rules out collapse, how far a plan error can travel into tokens or actions, and how to set the uncertainty gate; each was also checked numerically (Section 4.6).

Assumptions. All three are conditional statements whose constants must be estimated, not certified (Section 5.6).

- **A1 (realizer regularity).** On a bounded set containing every state and plan reached during sampling, the velocity $v(x, t, Z)$ is L_x-Lipschitz in x and L_z-Lipschitz in Z, uniformly in t. Spectral normalization encourages this, but dot-product attention is not globally Lipschitz [44].
- **A2 (encoder regularity).** On the data manifold, the encoder satisfies $\mu_E\|y - y'\| \le \|f(y) - f(y')\| \le L_E\|y - y'\|$ for task-relevant states y. As in VPA, the lower constant depends on the relation between observations and physical state and cannot be enforced by architecture alone [4].
- **A3 (latent dynamics).** The latent transition g is L_g-Lipschitz, and the one-step model error satisfies $\|z_{k+1} - g(z_k)\| \le \eta$.

4.1 Why the plan must carry a mode

Proposition 1 (factorization error of a parallel step)**.** *Let $q(w \mid c)$ be any distribution on $\mathcal{V}^N$. Over all factorized $r(w) = \prod_i r_i(w^i)$,*

$$\min_{r \text{ factorized}} \mathrm{KL}\left(q(\cdot \mid c) \,\|\, r\right) = \mathrm{TC}(W \mid c) := \sum_{i=1}^{N} H\left(W^i \mid c\right) - H\left(W \mid c\right), \tag{22}$$

attained at the product of marginals. (a) Conditioning on a deterministic function $Z = g(c)$ leaves the total correlation unchanged. (b) If $q(w \mid c) = \sum_u \pi(u \mid c)\, q_u(w \mid c)$ with every q_u factorized, then $\mathrm{TC}(W \mid c, U) = 0$, while

$$0 \le \mathrm{TC}(W \mid c) \le \sum_{i=1}^{N} I\left(W^i; U \mid c\right) \le N\, H(U \mid c). \tag{23}$$

Proof. $\mathrm{KL}(q\|r) = -H(W) - \sum_i \mathbb{E}_q \log r_i(W^i) = \mathrm{TC} + \sum_i \mathrm{KL}(q_i\|r_i)$, minimized at $r_i = q_i$. (a) holds because $q(\cdot \mid c, g(c)) = q(\cdot \mid c)$. (b) Given U, W factorizes, so $H(W \mid c, U) = \sum_i H(W^i \mid c, U)$ and the conditional total correlation is zero; and $H(W \mid c) \ge H(W \mid c, U)$ gives $\mathrm{TC}(W \mid c) \le \sum_i [H(W^i \mid c) - H(W^i \mid c, U)]$. □

Part (a) is the reason the planner samples a mode. A deterministic plan, however good a regression, cannot reduce the error of one factorized step; it can only make the posterior easier for a finite network to compute. A sampled mode removes the between-mode part of the error. The effect is large in the simplest case: if N positions copy one fair bit (the partition-factor and II pair of Section 1.2 is the case $N = 2$), $\mathrm{TC} = (N - 1)\log 2$ without the mode and 0 with it. In a contract, the choice between a mutual and a one-way confidentiality agreement is such a mode.

4.2 Representation regularity under VICReg

Theorem 1 (no rank-one collapse)**.** *Let $C = C(Z)$ be the sample covariance of $Z \in \mathbb{R}^{n\times d}$. If $v(Z) = 0$ and $c(Z) < (\gamma^2 - \epsilon)^2/(d(d-1))$, then*

$$\lambda_{\min}(C) \;\geq\; \gamma^2 - \epsilon - \sqrt{d(d-1)\,c(Z)} \;>\; 0. \tag{24}$$

Hence C is positive definite, $\operatorname{rank}(C) = d$, and the batch has collapsed neither to a point nor to a line. Because $\operatorname{rank}(C) \leq n-1$, the premise also forces $n \geq d+1$.

Proof. $v(Z) = 0$ means $\sqrt{C_{jj}+\epsilon} \geq \gamma$, so $C_{jj} \geq \gamma^2 - \epsilon$ for every j. By Gershgorin's theorem, $\lambda_{\min}(C) \geq \min_j (C_{jj} - R_j)$ with $R_j = \sum_{k\neq j} |C_{jk}|$. By Cauchy–Schwarz,

$$R_j \leq \sqrt{d-1}\,\Big(\sum_{k\neq j} C_{jk}^2\Big)^{1/2} \leq \sqrt{d-1}\,\Big(\sum_{k\neq l} C_{kl}^2\Big)^{1/2} = \sqrt{d-1}\,\sqrt{d\,c(Z)}.$$

□

The threshold is a worst case of order $1/d^2$, which finite batches rarely reach, since sampling noise alone puts off-diagonal entries of order $1/\sqrt{n}$. In practice the penalties stay positive, and the graded statement below is the relevant one.

Proposition 2 (stable rank)**.** *For any Z,*

$$\operatorname{rank}(C) \;\geq\; \frac{(\operatorname{tr} C)^2}{\|C\|_F^2} = \frac{\Big(\sum_j C_{jj}\Big)^2}{\sum_j C_{jj}^2 + d\,c(Z)}, \tag{25}$$

and if every variance equals s^2, then $\operatorname{rank}(C) \geq d/(1 + c(Z)/s^4)$.

Proof. For the r nonzero eigenvalues of the positive semidefinite C, Cauchy–Schwarz gives $(\sum \lambda)^2 \leq r \sum \lambda^2$, with $\sum \lambda = \operatorname{tr} C$ and $\sum \lambda^2 = \|C\|_F^2 = \sum_j C_{jj}^2 + d\,c(Z)$. □

Lowering $c(Z)$ at fixed variance therefore raises a guaranteed number of active dimensions continuously; $c(Z)/s^4 = 0.1$ guarantees at least $0.91\,d$.

Proposition 3 (gradient of the variance hinge)**.** *For a dimension j whose hinge is active, the gradient with respect to the column $z^j \in \mathbb{R}^n$ has norm*

$$\Big\|\nabla_{z^j}\Big(\gamma - \sqrt{\operatorname{Var}(z^j)+\epsilon}\Big)\Big\|_2 = \sqrt{\frac{\operatorname{Var}(z^j)}{(n-1)\,(\operatorname{Var}(z^j)+\epsilon)}}, \tag{26}$$

which tends to $1/\sqrt{n-1}$ when $\operatorname{Var}(z^j) \gg \epsilon$. A hinge on the variance itself, $\max(0, \gamma^2 - \operatorname{Var}(z^j))$, has gradient norm $2\sqrt{\operatorname{Var}(z^j)/(n-1)}$, which vanishes as the dimension collapses; the ratio of the two is $1/(2\sqrt{\operatorname{Var}(z^j)+\epsilon})$. Inside $v(Z)$ both carry the factor $1/d$.

Proof. $\partial \operatorname{Var}(z^j)/\partial z_i^j = 2(z_i^j - \bar{z}^j)/(n-1)$, because the mean's own dependence cancels when summed against centered values. The chain rule gives components $-(z_i^j - \bar{z}^j)/\big((n-1)\sqrt{\operatorname{Var}+\epsilon}\big)$, whose squared sum is $\operatorname{Var}/\big((n-1)(\operatorname{Var}+\epsilon)\big)$. □

The standard-deviation hinge therefore pushes a shrinking dimension apart with a gradient of fixed size until its spread approaches $\sqrt{\epsilon}$. With $\epsilon = 10^{-4}$, the gradient keeps 99.5% of its asymptotic size at $\operatorname{Var} = 10^{-2}$ and 71% at $\operatorname{Var} = 10^{-4}$. This sharpens a common claim: the gradient does not vanish *above the scale set by* ϵ, not at exact collapse, so ϵ must stay well below γ^2.

4.3 Validity of the energy-tilted kernel

Proposition 4. *Let $\tilde{K}^i$ be the kernel of Section 3.3, with the safety normalization*

$$\tilde{K}^i(j) = \frac{\pi_{t,h}\,\tilde{\rho}^{\,i}(j)}{\max\left(1, \sum_{j' \neq a} \tilde{\rho}^{\,i}(j')\right)}, \qquad j \neq a.$$

(a) For every $T > 0$, every $E \geq 0$ and every $c_{\max} \in (0, 1]$, $\tilde{K}^i$ is a probability distribution on $\mathcal{V}$, and the normalization is inactive.

(b) The probability of staying is at least $1 - \pi_{t,h}$.

(c) For the repair-aware multiplier $e^{-[E(j)-E(a)]/T}$, which can exceed one, $\tilde{K}^i$ remains a probability distribution.

(d) Let the energies be plan-mediated *and hard: $E^i(j \mid w^{-i}, Z) = E^i(j \mid Z) \in \{0, \infty\}$, with admissible sets $\mathcal{A}_i(Z) = \{j : E^i(j \mid Z) = 0\}$. If every position of w_t is admissible, every position of w_{t+h} is admissible under the fully parallel update. This can fail when E depends on w^{-i}.*

Proof. (a) For $j \neq a$, $e^{-E/T} \leq 1$ and $\min(\cdot, c_{\max})$ does not increase its argument, so $\sum_{j \neq a} \tilde{\rho}^{\,i}(j) \leq \sum_{j \neq a} p_{1|t}(j) \leq 1$; the jump mass is at most $\pi_{t,h} \leq 1$ and the staying mass is non-negative. (b) follows from the same inequality. (c) holds by construction of the normalization. (d) A token with infinite energy receives $\tilde{\rho} = 0$, so every jump lands in $\mathcal{A}_i(Z)$ and a position that does not jump keeps an admissible token; since the admissible sets do not depend on other positions, the conjunction holds after a simultaneous update. For the failure case, take the draft-dependent constraint $w^1 + w^2 \leq 1$ on two bits at the state $(0, 0)$: each position may move to 1 given the other's current value, and a simultaneous move reaches the inadmissible $(1, 1)$. □

Part (d) is the formal reason to route constraints through the plan. Draft-dependent constraints remain useful as soft energies, but their hard guarantee needs sequential updates or a final repair pass.

4.4 Error propagation from plan to output

Theorem 2 (realizer sensitivity)**.** *Under A1, run K Euler steps of size $1/K$ from the same initial state x_0 under plans Z and Z'. Then*

$$\|x_K - x'_K\| \;\leq\; \frac{L_z}{L_x}\left[\left(1 + \frac{L_x}{K}\right)^K - 1\right]\|Z - Z'\| \;\leq\; \frac{L_z}{L_x}\left(e^{L_x} - 1\right)\|Z - Z'\|. \tag{27}$$

Proof. Let $e_k = \|x_k - x'_k\|$ and $\Delta = \|Z - Z'\|$. Then $e_{k+1} \leq e_k + (L_x e_k + L_z \Delta)/K$ with $e_0 = 0$, so $e_K \leq (L_z \Delta/K) \sum_{j=0}^{K-1} (1 + L_x/K)^j = (L_z/L_x)[(1 + L_x/K)^K - 1]\,\Delta$, and $(1 + x/K)^K \leq e^x$. □

For $L_x = 1$, the factor multiplying L_z is 1.00, 1.25, 1.44, 1.57 and 1.64 at $K = 1, 2, 4, 8, 16$, against the continuous-time value $e - 1 \approx 1.72$. Few-step integration thus does not loosen the worst-case sensitivity to the plan; it trades integration accuracy, not robustness. The theorem covers the Logit-KL realizer ($x = l$) and VPA's action solver ($x = A$). When the constants depend on time, as the factor $1/(1 - t)$ in the Logit-KL velocity does, the same recursion gives $\|x_K - x'_K\| \leq \Delta \sum_k h\, L_z(t_k) \prod_{m>k} (1 + h\, L_x(t_m))$; on the uniform grid $h/(1 - t_k) \leq 1$, so no step is evaluated at the singularity.

Corollary 1 (token-invariance margin)**.** *For the Logit-KL realizer with shared source noise l_0, let $m^i = l^i_K(\hat{w}^i) - \max_{j \neq \hat{w}^i} l^i_K(j)$ be the top-two margin at position i under Z, and B the factor of Theorem 2. If $m^i > 2B\|Z - Z'\|$, position i decodes to the same token under Z'.*

Proof. Every logit moves by at most $\|l_K - l'_K\|_\infty \leq \|l_K - l'_K\|_2 \leq B\|Z - Z'\|$, so the margin shrinks by at most twice that. □

Proposition 5 (CTMC coupling bound)**.** *Suppose the posterior is Lipschitz in the plan in ℓ_1, $\|p_{1|t}(\cdot \mid w, Z) - p_{1|t}(\cdot \mid w, Z')\|_1 \leq L_p\|Z - Z'\|$ for all w, t, and the energies do not depend on the plan. Run Algorithm 1 under Z and Z' with a maximal coupling of each position's kernel at each step. Then*

$$P\left(\hat{w}_Z \neq \hat{w}_{Z'}\right) \;\leq\; N\, L_p\, \|Z - Z'\| \sum_{k=0}^{K-1} \pi_{t_k,\,1/K}, \tag{28}$$

and for the linear schedule the sum equals the harmonic number $\mathsf{H}_K = \sum_{k=1}^{K} 1/k$ *(for example* $\mathsf{H}_8 \approx 2.72$*).*

Proof. While the chains agree, the two kernels at a position differ in total variation by at most $\pi_{t_k} \sum_{j \neq a} |p(j) - p'(j)| \leq \pi_{t_k} L_p \|Z - Z'\|$: the map $x \mapsto \min(x\, e^{-E/T}, c_{\max})$ is 1-Lipschitz, and the staying mass absorbs the same differences. Under maximal coupling the chains first separate at step k and position i with at most this probability; a union bound over positions and steps gives the claim, and $\kappa_t = t$ gives $\pi_{t_k,1/K} = 1/(K-k)$. Plan-dependent energies add their own Lipschitz term. □

The bound grows only logarithmically in K but linearly in N, so it is informative per block rather than per file. It also shows that tilting and clipping never increase sensitivity to the plan.

Theorem 3 (horizon-independent containment)**.** *Assume A2–A3 and re-anchor the latent state by encoding a fresh observation at least every H steps, with encoding error at most δ_0. Then, for $L_g \neq 1$,*

$$\sup_t \|z_t - \hat{z}_t\| \;\leq\; \bar{e}(H) := L_g^H\, \delta_0 + \eta\, \frac{L_g^H - 1}{L_g - 1}, \tag{29}$$

independent of the total horizon; without re-anchoring the same recursion gives $L_g^t \delta_0 + \eta(L_g^t - 1)/(L_g - 1)$, which grows exponentially in t when $L_g > 1$. If, in addition, a milestone transition fires on the predicted-latent test $\|\hat{z}_t - z_g\| < \tau$ and $\|z_t - \hat{z}_t\| \leq \delta_\eta$, then the task-relevant state at the transition satisfies

$$\|y_t - y_g\| \;\leq\; \frac{\tau + \delta_\eta}{\mu_E}, \qquad \delta_\eta \leq \bar{e}(H), \tag{30}$$

and the test is certain to fire once $\|y_t - y_g\| < (\tau - \delta_\eta)/L_E$.

Proof. $e_{k+1} \leq \|g(z_k) - g(\hat{z}_k)\| + \eta \leq L_g e_k + \eta$; unroll from $e_0 \leq \delta_0$ and restart at each re-anchor, which comes after at most H steps. At a transition, $\|z_t - z_g\| \leq \|z_t - \hat{z}_t\| + \|\hat{z}_t - z_g\| < \delta_\eta + \tau$, and A2 gives $\mu_E \|y_t - y_g\| \leq \|z_t - z_g\|$. Conversely, $\|\hat{z}_t - z_g\| \leq \delta_\eta + L_E \|y_t - y_g\| < \tau$. □

With $\delta_\eta = 0$ the transition bound is VPA's τ/μ_E [4]; the theorem adds the prediction error and the horizon-independent bound on it. For long programs and contracts generated section by section, re-anchoring means re-encoding the committed draft and re-planning between sections, and y is the AST region structure or the clause outline.

4.5 Uncertainty-gated horizons

The executed horizon (for control) or the number of committed positions per re-planning round (for long programs and contracts) follows VPA's gate,

$$H_t = \max\left(H_{\min},\; \left\lfloor H_{\max}\, e^{-\beta\, \bar{\sigma}_{t+1}} \right\rfloor\right). \tag{31}$$

It is non-increasing in $\bar{\sigma}$, equals $H_{\max}$ at $\bar{\sigma} = 0$, and saturates at $H_{\min}$ for $\bar{\sigma} \geq \sigma_{\text{sat}} := \ln(H_{\max}/H_{\min})/\beta$; without the floor it is $\beta H_{\max}$-Lipschitz in $\bar{\sigma}$.

Theorem 4 (risk-calibrated gate)**.** *Suppose per-step latent errors within an executed chunk are zero-mean and uncorrelated with second moment at most $\bar{\sigma}^2$, so that $\mathbb{E}\|e_H\|^2 \leq H\bar{\sigma}^2$, and call a chunk failed if $\|e_H\| \geq \tau$. If*

$$\beta \;\geq\; \frac{2\sqrt{H_{\max}}}{e\, \tau\, \sqrt{\alpha}}, \tag{32}$$

then $P(\text{failure}) \leq \alpha$ for every $\bar{\sigma} < \sigma_{sat}$. For $\bar{\sigma} \geq \sigma_{sat}$ the guarantee holds only while $\bar{\sigma} \leq \tau\sqrt{\alpha/H_{\min}}$; beyond that the controller must escalate by re-observing every step, slowing down, or stopping.

Proof. Markov's inequality on $\|e_H\|^2$ gives $P(\|e_H\| \geq \tau) \leq H\bar{\sigma}^2/\tau^2$. Above saturation $H \leq H_{\max} e^{-\beta\bar{\sigma}}$, and $\sigma^2 e^{-\beta\sigma}$ is maximized at $\sigma = 2/\beta$ with value $4e^{-2}/\beta^2$, so the risk is at most $4H_{\max}/(e^2\beta^2\tau^2) \leq \alpha$. At saturation $H = H_{\min}$ and the risk is at most $H_{\min}\bar{\sigma}^2/\tau^2$. □

For $H_{\max} = 16$, $\tau = 1$ and $\alpha = 0.05$ the rule gives $\beta \geq 13.2$. If errors are correlated or the dynamics expand ($L_g > 1$), replace $H\bar{\sigma}^2$ by $\bar{\sigma}^2 \sum_{k<H} L_g^{2k}$; the argument is unchanged and yields a larger β. Markov's bound is conservative; the practical requirement is a calibrated $\bar{\sigma}$, which we obtain by split conformal calibration on held-out rollouts.

4.6 Numerical verification

Every bound above was checked against random instances; none was violated. Appendix A.1 describes each check, and Appendix B lists the script.

Table 2: Numerical verification of the theoretical results on random instances; no bound was violated.

Result	What was checked	Instances	Outcome
Theorem 1	$\lambda_{\min}(C)$ against the bound, random covariances with $v(Z) = 0$, $d = 32$	3,207 of 20,000 draws met the premise	No violation; smallest slack 0.0039
Proposition 2	$\mathrm{rank}(C)$ against stable rank; equal-variance formula	20,000 draws; 1 exact case	No violation; formula matched (23.0764)
Proposition 3	Gradient norm against the closed form, $n = 256$, Var from 10^{-1} to 10^{-6}	5	Matched to 4 significant digits
Theorem 2	Empirical $\|x_K - x'_K\|/\|Z - Z'\|$ against the bound, random tanh velocity fields, $K \in \{1, 2, 4, 8, 16\}$	2,500	Largest ratio 0.83
Lemma 1	Exact jump mass in $[0, 1]$ for linear, $\sqrt{t}$ and cosine schedules, $K \in \{2, 4, 8\}$	9	Exact mass always valid; Euler mass far above 1 for $\sqrt{t}$
Theorem 4	Maximum risk over $\bar{\sigma}$ at the minimal β	3 settings	Equal to α in each (0.05, 0.05, 0.10)
Lemma 2	Identity $l_1 - l_0 = (l_1 - l_t)/(1 - t)$	1	Error 8.9×10^{-16}

5 Experimental Protocol and Evaluation

No end-to-end PR-JEPA measurement exists yet. This section fixes the protocol before any run: every cell marked — will be filled by it, and every other number is either reported by the cited work or predicted by the roofline model of Section 5.4, and is labeled as such.

5.1 Training protocol

1. **Stage 1, planner.** Train f_θ, π^h_ϕ and g_ϕ with $\mathcal{L}_{\text{plan}}$, the target encoder following the EMA schedule from $m_0 = 0.996$ to 1. Modes u are extracted from the reference: directive classes parsed from the pragmas for HLS-C, imported libraries and clusters of AST shapes for general-purpose code, template variant and governing law from clause annotations for contracts, and VPA's primitive labels for control. After each epoch we log $v(Z)$, $c(Z)$ and the stable rank of Proposition 2, and stop a run whose stable rank falls below $d/2$.
2. **Stage 2, realizer.** Freeze the planner and train F_ψ with $\mathcal{L}_{\text{real}}$, conditioning on the plan the frozen planner *predicts* from c, never on the target encoder's $f_{\bar{\theta}}(y)$. Conditioning on the target encoder's view leaks the answer into the plan and opens a train–test gap of the same kind as exposure bias. We use FS-DFM's step-budget conditioning, distillation from a 1,024-step teacher, and plan dropout 0.1.
3. **Stage 3 (optional), joint fine-tuning.** Unfreeze the planner at one tenth of the learning rate with its regularizers kept on.

Why not end-to-end from the start. Three risks argue for the staged protocol. Realizer gradients pull Z toward token-predictive features, which can undo the VICReg geometry and break the premise of Theorem 1. If the planner sees the target during joint training, Z can encode the answer verbatim and the realizer learns to copy it. And the realizer would chase a conditioning distribution that moves under it, which is why VPA freezes its predictor before training its solver [4]. Ablation A9 measures the second risk directly.

Data. General-purpose code: permissively licensed source files with ASTs from tree-sitter for training; HumanEval, MBPP and MultiPL-E for evaluation, and MBPP with 10%, 50% and 90% of lines removed for infilling, as in Sevriugov et al. [14]. HLS-C: the roughly 16.7K verified HLS-C programs of SAGE-HLS [5], with ASTs from Clang, and its VerilogEval-derived problems as the test set. Legal documents: contracts from SEC EDGAR filings, with clause outlines, defined terms and cross-references extracted by rule and checked against the CUAD and LEDGAR clause labels [39, 40]; each test item is a term sheet (contract type, parties, dates, amounts, required clause categories, governing law) built from a held-out contract. Text: 1,024-token generation under the FS-DFM protocol [13]. Control: the LIBERO-10 demonstrations [45]. Every training corpus is deduplicated against every evaluation set.

5.2 Tasks and metrics

- **Task A, HLS-C generation.** Input: a natural-language specification and a function signature. Metrics: Synth@1 (C synthesis completes in AMD Vitis HLS 2024 for the Zynq UltraScale+ XCZU3EG target used by SAGE-HLS); functional Pass@1 and Pass@10 under SAGE-HLS's constrained RTL simulation, with the unbiased Pass@k estimator [34]; bit-width consistency $\mathrm{BWC} = 1 - n_{\text{trunc}}/n_{\text{assign}}$ from a Clang-based checker, where n_{assign} counts assignments and n_{trunc} those with implicit truncation or extension; directive legality (fraction of pragmas accepted without warnings); quality of results on passing designs (latency in cycles, achieved II, LUT, FF, DSP and BRAM); and generation latency.
- **Task B, general-purpose code.** Input: a docstring and signature (HumanEval, MBPP) or a program with lines removed (MBPP infilling). Metrics: Pass@1 and Pass@10 on HumanEval and MBPP in Python and on MultiPL-E in C++, Java and Rust; Compiles@1 for infilling; and two static measures of global consistency, the rate of undefined or unused names (pyflakes for Python, compiler diagnostics for the other languages) and the type-check pass rate (mypy on annotated Python).
- **Task C, legal documents.** Input: a term sheet; output: a complete contract of the requested type. Automatic metrics: defined-term consistency (every capitalized defined term is defined exactly once and used at least once, and no undefined capitalized term appears); cross-reference validity (every reference such as "Section 9.2" resolves to an existing section); entity and number consistency (party names, dates and amounts match the term sheet); clause coverage (share of required clause categories found by a classifier trained on LEDGAR and CUAD labels); and internal contradiction rate (clause pairs that an NLI model trained on ContractNLI labels as contradictory). Human evaluation: licensed attorneys, blind to the system, rate 100 contracts per system for completeness, clarity and legal soundness on a 1–5 scale. These metrics measure internal consistency and coverage, not enforceability; outputs are drafts for attorney review, not legal advice. As a domain-general check, unconditional 1,024-token text is scored by generative perplexity and token entropy under the FS-DFM protocol.
- **Task D, embodied control.** LIBERO-10 in the VPA configuration ($K \in \{1, 2, 3\}$, $H = 16$) with the PR-JEPA bridge and the calibrated gate. Metrics: success over 200 episodes with 95% Wilson intervals; decision-step latency (median and 95th percentile, batch 1) on an Apple M4 and on an embedded GPU; and reward retention under one-step integration, the ratio of success at $K = 1$ to success at $K = 16$.

Statistics. Three seeds per configuration; bootstrap 95% intervals for Pass@k; latency as median and 95th percentile over 1,000 runs after 50 warm-up runs, at batch 1 in bf16, with every method on the same hardware.

5.3 Pre-registered hypotheses

- **H1, latency.** At $N = 1{,}024$ on the same GPU, PR-JEPA with $K = 8$ has at least $10\times$ lower median latency than AR decoding with a backbone of the same size; the roofline model predicts $17\times$.
- **H2, consistency.** At equal K, PR-JEPA improves the plan-mediated consistency metrics over FS-DFM conditioned on c alone: BWC and directive legality for HLS-C, the undefined-name rate for general-purpose code, and defined-term consistency and cross-reference validity for contracts (Proposition 4(d)).
- **H3, modes.** Removing the mode selector raises the rate of mode-mixing errors: directive combinations absent from every reference design, and contracts that combine clauses from incompatible template variants (Proposition 1).
- **H4, geometry.** Setting $\lambda_c = 0$ lowers the stable rank of Z (Proposition 2) and lowers Pass@1, most at small K.
- **H5, parity on correctness.** Synth@1 lies within 5 points of SAGE-HLS's 92.95%, and HumanEval Pass@1 within 5 points of a masked-diffusion model of the same size. We do not hypothesize higher functional correctness than AR models fine-tuned on the same data.
- **H6, contracts.** In blinded attorney review, PR-JEPA drafts are rated no worse than drafts from an AR model of the same size given the same term sheet, while cross-reference validity and the absence of undefined terms, enforced as hard plan-mediated constraints, are near 100%.
- **H7, control.** The bridge and the calibrated gate keep VPA's decision-step latency within 20% without lowering LIBERO-10 success.

5.4 Analytical cost model

A roofline model predicts that NAR realization wins on single-request latency by 9–21× at $K = 8$ and loses on saturated throughput by about a factor of K. The model assumes NVIDIA H100 SXM peak figures (3.35 TB/s HBM bandwidth, 989 TFLOP/s dense bf16)[1], of which 70% of bandwidth and 50% of compute are achieved, and a 512-token prompt. AR decoding is a compute-bound prefill followed, per token, by the larger of weight-plus-cache read time and compute time. The NAR realizer runs K full passes over prompt plus output. The 7.6B row uses the public Qwen2.5-Coder-7B configuration [46] (28 layers, width 3,584, 4 KV heads of width 128); the 1.3B row assumes a multi-head configuration (24 layers, width 2,048, 16 KV heads of width 128) at FS-DFM's scale. Planner and bridge passes are small and not modeled. These are predictions, not measurements. Appendix A.2 gives the formulas and the script that computes Table 3.

Table 3: Predicted single-request latency (batch 1) and AR KV cache.

Backbone	Output tokens N	AR latency (ms)	NAR, K = 4 (ms)	NAR, K = 8 (ms)	AR ÷ NAR at K = 8	AR KV cache per sequence (MiB)
7.6B, GQA	4,096	26,822	636	1,271	21.1×	252
7.6B, GQA	1,024	6,679	197	393	17.0×	84
7.6B, GQA	256	1,679	96	193	8.7×	42
1.3B, MHA	4,096	5,423	131	261	20.7×	864
1.3B, MHA	1,024	1,226	36	72	17.0×	288
1.3B, MHA	256	300	17	34	8.8×	144

The advantage grows with N because each AR token pays a full weight read, while each NAR pass processes every token at once. Throughput reverses the picture. When both are compute-bound, the NAR realizer spends K times the arithmetic per sample, so its peak throughput is about $1/K$ of AR's. Under the stated efficiencies AR decoding becomes compute-bound at a batch of about 211, independent of model size. Whether AR can reach that batch depends on its cache: on an 80 GB device the 7.6B GQA model fits 736 sequences at $N = 1{,}024$ and 130 at $N = 8{,}192$, the 1.3B multi-head model 256 and 45. For long outputs from multi-head models, AR therefore stays memory-bound and the throughput gap narrows. PR-JEPA is a latency architecture, not a throughput one.

5.5 Comparison tables

Reported values come from the cited papers and use their harnesses; our runs must reproduce those harnesses before any comparison. Tables 4–7 fix the comparisons for Tasks A–D.

Table 4: Task A, HLS-C generation ($N \approx 1{,}024$ tokens; latency at batch 1). †Reported in the cited paper.

Method	Paradigm	Latency (ms)	Throughput (tokens/s)	Peak VRAM (GB)	Synth@1 (%)	Pass@1 (%)	BWC (%)
Qwen2.5-Coder-7B, base [5]	AR	—	—	—	52.56†	22.44†	—
SAGE-HLS, AST-guided fine-tune [5]	AR	—	—	—	92.95†	57.69†	—
DFM, conditioned on c	NAR, 1,024 steps	—	—	—	—	—	—
FS-DFM, conditioned on c	NAR, $K = 8$	—	—	—	—	—	—
Logit-KL, conditioned on c	NAR, $K = 8$	—	—	—	—	—	—
Pure JEPA (plan + per-position decoder)	NAR, $K = 1$	—	—	—	—	—	—
PR-JEPA (FS-DFM realizer)	NAR, $K = 8$	—	—	—	—	—	—
PR-JEPA (Logit-KL realizer)	NAR, $K = 8$	—	—	—	—	—	—

SAGE-HLS's figures come from a QLoRA-fine-tuned 7B model; its Pass@10 is 75.64%. "Pure JEPA" decodes the plan with one parallel classifier pass and no refinement, which isolates the contribution of the realizer.

The reported rows differ in model size, data and decoding and are not directly comparable. At 10% infilling, Logit-KL's Pass@10 and Compiles@1 are 29.2% and 73.7% against 25.5% and 39.7% for DFM; FS-DFM reports perplexity parity with a 1,024-step DFM baseline for 1,024-token text [13].

[1] `https://www.nvidia.com/en-us/data-center/h100/`

Table 5: Task B, general-purpose code (Pass@1, %; latency at batch 1 for a 512-token program). †Reported in the cited paper.

Method	Paradigm	Latency (ms)	HumanEval	MBPP	MultiPL-E, C++/Java/Rust mean	MBPP infilling, 10% of lines	Undefined-name rate (%)
Qwen2.5-Coder base (as reported by Gong et al. [6])	AR	—	61.6†	75.9†	—	—	—
DiffuCoder-7B base [6]	Masked diffusion	—	67.1†	74.2†	—	—	—
DFM, 1.7B [12]	NAR flow	—	6.7†	6.7, 1-shot†	—	—	—
DFM [14]	NAR flow	—	—	—	—	11.1†	—
Logit-KL, best inference scheme [14]	NAR flow	—	—	—	—	17.4†	—
FS-DFM, conditioned on c	NAR, $K = 8$	—	—	—	—	—	—
PR-JEPA (FS-DFM realizer)	NAR, $K = 8$	—	—	—	—	—	—
PR-JEPA (Logit-KL realizer)	NAR, $K = 8$	—	—	—	—	—	—

Table 6: Task C, contract drafting from term sheets (automatic metrics in %; attorney rating 1–5).

Method	Latency (ms)	Defined-term consistency	Cross-reference validity	Entity and number consistency	Clause coverage	Contradiction rate	Attorney rating
AR model of the same size, term sheet in the prompt	—	—	—	—	—	—	—
FS-DFM, conditioned on c, $K = 8$	—	—	—	—	—	—	—
PR-JEPA (FS-DFM realizer), $K = 8$, section by section	—	—	—	—	—	—	—
PR-JEPA without plan-mediated energies ($T = \infty$)	—	—	—	—	—	—	—

No public benchmark scores contract drafting on these axes, so every row is to be measured.

Table 7: Task D, LIBERO-10 control. †Reported in the cited paper.

Method	Decision-step latency	Hardware	Success (%), 200 episodes	Retention, $K = 1$ vs $K = 16$
RT-2, 55B [3] via Lee [4]	About 1–3 Hz†	—	—	—
VPA [4]	10.5–14.2 ms† (one camera)	Apple M4	40.0, interval 33.5–46.9† (two cameras)	—
VPA + PR-JEPA bridge and calibrated gate	—	Apple M4	—	—

VPA's latency was measured for its single-camera configuration and its success rate for the two-camera one; the two numbers are not from one configuration, and neither is a controlled comparison with RT-2.

5.6 Ablations

Each ablation carries the effect that the theory predicts, so that a run can confirm or refute a specific claim.

6 Discussion, Limitations, and Conclusion

Separating planning from realization changes where errors live: global decisions are made once, in a regularized latent space with an explicit mode, and token-level work becomes conditional translation whose sensitivity to the plan is bounded.

Table 8: Ablations, predicted effects, and results.

ID	Change	Predicted effect	Source of prediction	Result
A1	Step budget $K \in \{1, 2, 4, 8, 16\}$	Latency grows as $K + 3$; quality saturates near $K = 8$	Section 3.3; FS-DFM	—
A2	No energy barrier ($T = \infty$)	Lower plan-mediated consistency (BWC, undefined names, defined terms, cross-references); latency unchanged	Proposition 4	—
A3	Hard barrier ($T \to 0$)	Plan-mediated constraints met by construction; positions may stall when the posterior favors inadmissible tokens	Proposition 4(d)	—
A4	Repair-aware tilt	Fewer unrepaired violations from inadmissible drafts	Section 3.3	—
A5	Euler kernel instead of the exact kernel	Invalid-kernel events for non-linear schedules; lower quality at $K \leq 4$	Lemma 1	—
A6	No covariance penalty ($\lambda_c = 0$)	Lower stable rank of Z; Pass@1 drop largest at small K	Proposition 2	—
A7	No variance penalty ($\lambda_v = 0$)	Dimensional collapse; premise of Theorem 1 fails	Theorem 1	—
A8	No mode selector	More mode-mixing errors	Proposition 1	—
A9	Realizer trained on target-encoder plans	Train–test gap; lower test Pass@1	Section 5.1	—
A10	Gate β below the Theorem 4 minimum	Higher chunk-failure rate in control	Theorem 4	—
A11	Estimate L_x, L_z (Jacobian norms along sampling paths) and L_E, μ_E (finite differences over paired observations)	Tests assumptions A1–A2 and the tightness of Theorems 2–3	Section 4	—

6.1 What the decoupling buys

- **A divided error budget.** Proposition 1 splits the error of a parallel step into a between-mode part, which the sampled mode removes, and a within-mode part, which the plan and the refinement steps must handle. Each part has its own diagnostic: selector accuracy for the mode, stable rank for the plan, and the constants of Theorem 2 for the realizer.
- **Reusable plans.** One plan supports many realizations. Best-of-n selection against a synthesis or simulation check costs nK realizer passes but only one planning pass, and diversity comes from the source noise and from the mode.
- **Parallel-safe constraints.** Constraints routed through the plan survive fully parallel updates (Proposition 4(d)). This is a design rule for any NAR generator of structured text, whether HLS-C, Python, or a contract.
- **One pattern for tokens and actions.** The same planner, bridge and realizer cover discrete programs and continuous action chunks; VPA is the continuous instance.

6.2 Engineering trade-offs

- **Temperature T and clip $c_{\max}$.** A small T makes constraints hard but can stall a position whose posterior favors inadmissible tokens; Proposition 4(b) gives the staying probability. A large T recovers the untilted sampler. We tune T on a validation Pareto front of Synth@1 against generative perplexity, starting with hard plan-mediated terms and soft draft-dependent terms. A clip $c_{\max} < 1$ caps the jump probability of any single destination; it is required only for repair-aware tilting or extrapolating guidance, and we default to $c_{\max} = 1$ otherwise.
- **Schedule κ_t.** The exact kernel removes the Euler validity condition, but the schedule still allocates jump mass across steps. With the linear schedule the last step redraws every position, and the total jump mass is the harmonic number H_K (Proposition 5).
- **Choice of realizer.** The CTMC realizer stores N integers, is stochastic, and takes energies as simple multipliers. The Logit-KL realizer stores $N \times S$ numbers, about 0.3 GB at $N = 1{,}024$ with a 152K vocabulary, but is deterministic, admits margins (Corollary 1), and suits smaller vocabularies such as tokenized code; contracts, with natural-language vocabularies and long outputs, default to the CTMC realizer.

- **Throughput.** At saturation the realizer delivers about $1/K$ of AR throughput (Section 5.4). PR-JEPA belongs where latency matters, in interactive design loops and control; AR remains preferable for high-batch offline generation.
- **Calibration.** The gate is only as good as $\bar{\sigma}$, and the bounds only as good as their constants; ablation A11 estimates both.

6.3 Limitations

- No end-to-end PR-JEPA measurement exists yet; every PR-JEPA cell in Section 5 is pending.
- The theorems are conditional on Lipschitz constants that cannot be certified for transformers (A1) and on a co-Lipschitz constant that architecture cannot enforce (A2).
- Proposition 5 grows linearly in N and is informative per block, not per file.
- If mode slots are strongly coupled, the selector reintroduces a short sequential component, and mode vocabularies must be designed for each domain; a stochastic latent predictor in the style of Huo et al. [21] is an alternative where modes are hard to enumerate.
- Grammar energies over incomplete drafts require incremental parsers that tolerate holes. Full synthesis takes minutes per design, too slow for the inner loop, so it enters only as a final filter.
- The HLS-C corpus is small (about 16.7K programs), and AST-structured targets assume reference code that parses.
- The risk bound of Theorem 4 uses Markov's inequality and is conservative.
- Contract drafting has no public generation benchmark. Our legal metrics measure internal consistency and clause coverage, not enforceability or fitness for a client's purpose, and the corpora we rely on are mostly English-language U.S. securities filings; other jurisdictions and languages, including Korean law, need their own corpora and attorney reviewers.
- For code, plan-mediated constraints check declared names, signatures and imports, not program semantics; unit tests remain the arbiter of correctness.

6.4 Broader impact

Real-time edge AI. A sequential depth that does not grow with the horizon makes reactive control feasible on small devices. Lower latency does not by itself make a controller safe, and the escalation rule of Theorem 4 (re-observe, slow down, or stop when uncertainty saturates the gate) should be part of any deployment.

Code and hardware synthesis. Faster generation of candidate programs and designs speeds code completion and design-space exploration. Generated software and hardware can carry subtle functional or security errors, so tests, synthesis, simulation and formal checks remain mandatory before deployment or fabrication.

Legal documents. Faster drafting can lower the cost of routine agreements, but a contract can be internally consistent and still wrong for the parties' situation or jurisdiction, and AI legal tools still hallucinate [7]. PR-JEPA's outputs are drafts for review by a qualified lawyer, not legal advice, and a licensed attorney should remain responsible for every document that is signed.

Agent design. Plan-then-realize separates a slow global decision from fast parallel execution. Latent plans are harder to inspect than text plans; the discrete mode u gives a short, human-readable summary of what the planner decided.

6.5 Conclusion

PR-JEPA plans once and realizes in parallel, with one architecture for general-purpose code, HLS-C, legal documents and control. A JEPA planner, kept full-rank by VICReg and an EMA target, samples a design mode and predicts a continuous plan; a cross-attention bridge hands that plan to a realizer that refines every position in K steps, through an exact, energy-tilted and clipped jump kernel or a straight path in logit space. The analysis explains why the plan must carry a mode, when VICReg excludes collapse, how far plan errors can travel, how re-anchoring keeps them bounded over any horizon, and how to set the uncertainty gate for a target risk. The cost model predicts large latency gains and a throughput penalty of about K, which places the method in latency-critical settings. The next step is to run the protocol of Section 5 and fill its tables.

References

[1] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, et al. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2023. arXiv:2211.05102.

[2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 2023. arXiv:2309.06180.

[3] Anthony Brohan et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, 2023. arXiv:2307.15818.

[4] Yeonseok Lee. Vision-predictor-action: Towards efficient embodied AI via neuro-symbolic JEPA predictors. *arXiv preprint arXiv:2610.15556*, 2026.

[5] M. Zafir Sadik Khan, Nowfel Mashnoor, Mohammad Akyash, Kimia Azar, and Hadi Kamali. SAGE-HLS: Syntax-aware AST-guided LLM for high-level synthesis code generation. *arXiv preprint arXiv:2508.03558*, 2025.

[6] Shansan Gong, Ruixiang Zhang, Huangjie Zheng, Jiatao Gu, Navdeep Jaitly, Lingpeng Kong, and Yizhe Zhang. DiffuCoder: Understanding and improving masked diffusion models for code generation. *arXiv preprint arXiv:2506.20639*, 2025.

[7] Varun Magesh, Faiz Surani, Matthew Dahl, Mirac Suzgun, Christopher D. Manning, and Daniel E. Ho. Hallucination-free? Assessing the reliability of leading AI legal research tools. *Journal of Empirical Legal Studies*, 2025. arXiv:2405.20362.

[8] Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2107.03006.

[9] Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2310.16834.

[10] Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, et al. Simple and effective masked diffusion language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2406.07524.

[11] Andrew Campbell, Jason Yim, Regina Barzilay, Tom Rainforth, and Tommi Jaakkola. Generative flows on discrete state-spaces: Enabling multimodal flows with applications to protein co-design. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.04997.

[12] Itai Gat, Tal Remez, Neta Shaul, Felix Kreuk, Ricky T. Q. Chen, Gabriel Synnaeve, Yossi Adi, and Yaron Lipman. Discrete flow matching. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2407.15595.

[13] Amin Karimi Monsefi, Nikhil Bhendawade, Manuel R. Ciosici, Dominic Culver, Yizhe Zhang, and Irina Belousova. FS-DFM: Fast and accurate long text generation with few-step diffusion language models. *arXiv preprint arXiv:2509.20624*, 2025.

[14] Egor Sevriugov, Nikita Dragunov, Anton Razzhigaev, Andrey Kuznetsov, and Ivan Oseledets. Logit-KL flow matching: Non-autoregressive text generation via sampling-hybrid inference. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2411.16821.

[15] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. arXiv:2301.08243.

[16] Adrien Bardes, Quentin Garrido, Jean Ponce, et al. Revisiting feature prediction for learning visual representations from video. *arXiv preprint arXiv:2404.08471*, 2024.

[17] Mido Assran et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.

[18] Jean-Bastien Grill, Florian Strub, Florent Altché, et al. Bootstrap your own latent: A new approach to self-supervised learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv:2006.07733.

[19] Adrien Bardes, Jean Ponce, and Yann LeCun. VICReg: Variance-invariance-covariance regularization for self-supervised learning. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2105.04906.

[20] Dengsheng Chen, Jie Hu, Xiaoming Wei, and Enhua Wu. Denoising with a joint-embedding predictive architecture. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.03755.

[21] Yanchen Huo, Ziying Song, and Yadan Luo. Flow-JEPA: Robust latent dynamics for JEPA world models via flow matching. *arXiv preprint arXiv:2608.29029*, 2026.

[22] Yang Zhang, Jiangyuan Zhao, Chenyou Fan, Jiayu Hu, Xiu Yuan, Chenjia Bai, and Xiu Li. V-JEPA policy: Building effective world-action models on predictive visual latents. *arXiv preprint arXiv:2609.37250*, 2026.

[23] Yuchen Zhu, Chenyi Xu, Yulin Zhang, Gang Xu, and Wentao Zhu. Juno: Taming predictive latents for vision-language-action models. *arXiv preprint arXiv:2610.09940*, 2026.

[24] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.02747.

[25] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2209.03003.

[26] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems (RSS)*, 2023. arXiv:2303.04137.

[27] Kevin Black, Noah Brown, Danny Driess, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[28] Ge Wang, Yibo Peng, Fan Feng, Shenhao Yan, Chengsi Yao, Jiahao Yang, Honghao Cai, Yiming Zhao, Xi Li, Jinke Ren, Shuguang Cui, Yatong Han, and Zhen Li. ReFPO: Reflow regularization for flow matching policy gradients. *arXiv preprint arXiv:2606.21086*, 2026.

[29] Shen Nie, Fengqi Zhu, Zebin You, et al. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025.

[30] Hunter Nisonoff, Junhao Xiong, Stephan Allenspach, and Jennifer Listgarten. Unlocking guidance for discrete state-space diffusion and flow models. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2406.01572.

[31] Jiatao Gu, James Bradbury, Caiming Xiong, Victor O. K. Li, and Richard Socher. Non-autoregressive neural machine translation. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1711.02281.

[32] Marjan Ghazvininejad, Omer Levy, Yinhan Liu, and Luke Zettlemoyer. Mask-predict: Parallel decoding of conditional masked language models. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019. arXiv:1904.09324.

[33] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*, 2023. arXiv:2211.17192.

[34] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[35] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

[36] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q. Feldman, Arjun Guha, Michael Greenberg, and Abhinav Jangda. MultiPL-E: A scalable and extensible approach to benchmarking neural code generation. *arXiv preprint arXiv:2208.08227*, 2022.

[37] Mingjie Liu, Nathaniel Pinckney, Brucek Khailany, and Haoxing Ren. VerilogEval: Evaluating large language models for Verilog code generation. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2023. arXiv:2309.07544.

[38] Haocheng Xu, Ye Qiao, Phyo Pyae Moe Aung, Alok Mishra, Pavana Prakash, Rolando Pablo Hong Enriquez, Adam Han Wu, Zhiheng Chen, Dejan Milojicic, and Sitao Huang. Structure-augmented LLMs for high-level synthesis pragma optimization. *arXiv preprint arXiv:2609.38601*, 2026.

[39] Dan Hendrycks, Collin Burns, Anya Chen, and Spencer Ball. CUAD: An expert-annotated NLP dataset for legal contract review. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021. arXiv:2103.06268.

[40] Don Tuggener, Pius von Däniken, Thomas Peetz, and Mark Cieliebak. LEDGAR: A large-scale multi-label corpus for text classification of legal provisions in contracts. In *Proceedings of the Twelfth Language Resources and Evaluation Conference (LREC)*, pages 1235–1241, 2020.

[41] Yuta Koreeda and Christopher D. Manning. ContractNLI: A dataset for document-level natural language inference for contracts. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, 2021. arXiv:2110.01799.

[42] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. arXiv:1612.01474.

[43] Takeru Miyato, Toshiki Kataoka, Masanori Koyama, and Yuichi Yoshida. Spectral normalization for generative adversarial networks. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1802.05957.

[44] Hyunjik Kim, George Papamakarios, and Andriy Mnih. The Lipschitz constant of self-attention. In *International Conference on Machine Learning (ICML)*, 2021. arXiv:2006.04710.

[45] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023. arXiv:2306.03310.

[46] Binyuan Hui et al. Qwen2.5-Coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.

A Reproducibility Script

The numerical verification of Section 4.6 (Table 2) and the analytical cost model of Section 5.4 (Table 3) are produced by one Python script, `pr_jepa_checks.py`, listed in full in Appendix B. It depends only on NumPy, fixes every random seed, and runs in about five seconds on a single CPU core. Each check prints the quantity it tests next to the value the theory requires, so a violated bound is visible directly in the output.

A.1 Checks of the theoretical results

(1) Gradient of the variance hinge (Proposition 3). For one embedding dimension with $n = 256$ samples, standardized and rescaled to variance $\mathrm{Var} \in \{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-6}\}$, the script computes the gradient of $\gamma - \sqrt{\mathrm{Var}(z) + \epsilon}$ with $\epsilon = 10^{-4}$ and compares its norm with the closed form $\sqrt{\mathrm{Var}\,/\big((n-1)(\mathrm{Var}+\epsilon)\big)}$. It also reports the ratio to the gradient of the variance itself, which equals $1/\big(2\sqrt{\mathrm{Var}+\epsilon}\big)$.

(2) Anti-collapse bound and stable rank (Theorem 1, Proposition 2). The script draws 20,000 symmetric 32×32 matrices C with diagonal entries in $[1, 2]$ and small random off-diagonal mass, and discards those that are not positive semidefinite. Whenever the premise of Theorem 1 holds (zero variance penalty, $v(Z) = 0$, with $\gamma = 1$), it records $\lambda_{\min}(C) - \big(\gamma^2 - \epsilon - \sqrt{d(d-1)\,c(Z)}\big)$, which must be non-negative. For every matrix it also checks $\mathrm{rank}(C) \geq \mathrm{tr}(C)^2/\|C\|_F^2$.

(3) Sensitivity of the Euler flow map (Theorem 2). The velocity field is $v(x, z) = \tanh(Ax + Bz)$ with $x \in \mathbb{R}^{16}$, $z \in \mathbb{R}^8$, and random A, B rescaled to spectral norms $L_x \in [0.2, 3]$ and $L_z \in [0.2, 2]$, which bound the Lipschitz constants. For 500 draws and $K \in \{1, 2, 4, 8, 16\}$ (2,500 cases), it integrates K Euler steps from a shared start under the plan z and a perturbed plan $z + \delta$ with $\|\delta\| \approx 10^{-3}$, and reports the largest ratio of $\|x_K - x'_K\|/\|\delta\|$ to the bound $(L_z/L_x)\big[(1 + L_x/K)^K - 1\big]$, which must not exceed 1.

(4) Exact few-step kernel versus Euler (Lemma 1). For the linear schedule $\kappa_t = t$, the square-root schedule $\kappa_t = \sqrt{t}$ and the cosine schedule $\kappa_t = 1 - \cos(\pi t/2)$, and for $K \in \{2, 4, 8\}$ (9 settings), it evaluates the exact jump mass $\pi_{t,h} = (\kappa_{t+h} - \kappa_t)/(1 - \kappa_t)$ on the grid $t = k/K$ and confirms that it lies in $[0, 1]$. It also reports the largest Euler jump mass $h\,\dot\kappa_t/(1 - \kappa_t)$, which exceeds 1 for the square-root schedule and is therefore not a valid probability.

(5) Harmonic sum of jump masses (Proposition 5). For the linear schedule it evaluates $\sum_{k=0}^{K-1} \pi_{t_k, 1/K} = \sum_{k=0}^{K-1} 1/(K-k) = H_K$ for $K \in \{1, 2, 4, 8, 16\}$.

(6) Risk-calibrated gate (Theorem 4). For $(H_{\max}, \tau, \alpha) \in \{(16, 1, 0.05), (32, 1, 0.05), (64, 1, 0.1)\}$ it sets $\beta = 2\sqrt{H_{\max}}/(e\,\tau\sqrt{\alpha})$ and maximizes the risk bound $H_{\max} e^{-\beta\bar\sigma}\bar\sigma^2/\tau^2$ over 2×10^5 values of $\bar\sigma \in [10^{-4}, 5]$; the maximum must equal α up to grid resolution and never exceed it.

(7) Logit-KL velocity identity (Lemma 2). With $S = 7$ classes, smoothing $\varepsilon = 0.05$ and $t = 0.6$, it builds the smoothed one-hot target $l_1 = a\,e_k + b\mathbb{1}$ with a and b as in Section 3.4, forms $l_t = (1 - t)\,l_0 + t\,l_1$ from a random l_0, and checks $l_1 - l_0 = (l_1 - l_t)/(1 - t)$ to machine precision.

A.2 Roofline cost model

Hardware and models. The model uses the vendor peak figures of an H100 SXM, $\mathrm{BW} = 3.35$ TB/s of HBM bandwidth and $F = 989$ TFLOP/s of dense BF16 compute, at assumed achieved fractions $\eta_{\mathrm{bw}} = 0.7$ and $\eta_F = 0.5$, so $\mathrm{BW}_e = \eta_{\mathrm{bw}}\mathrm{BW}$ and $F_e = \eta_F F$. Weights and cache entries take 2 bytes. Two backbones are modeled: a 7.6B model with the public Qwen2.5-Coder-7B configuration ($L = 28$ layers, width $d = 3{,}584$, $h_{kv} = 4$ KV heads of width $d_h = 128$) and a 1.3B model at FS-DFM's scale with an assumed multi-head configuration ($L = 24$, $d = 2{,}048$, $h_{kv} = 16$, $d_h = 128$). Every request carries a 512-token prompt.

Formulas. With P parameters, the KV cache costs $2\,L\,h_{kv}\,d_h \cdot 2$ bytes per token. Autoregressive (AR) decoding of N tokens is a compute-bound prefill followed by one step per token, each bounded by the larger of its memory and compute time:

$$T_{\mathrm{AR}} = \frac{2P \cdot 512}{F_e} + \sum_{i=0}^{N-1} \max\!\left(\frac{2P + \mathrm{KV}(512+i)}{\mathrm{BW}_e},\ \frac{2P + 4L\,(512+i)\,d}{F_e}\right), \tag{33}$$

where $\mathrm{KV}(n)$ is the cache size in bytes for n tokens. The non-autoregressive (NAR) realizer runs K full passes over $T = 512 + N$ positions:

$$T_{\mathrm{NAR}} = K \cdot \max\!\left(\frac{2P}{\mathrm{BW}_e},\ \frac{2PT + 4LT^2 d}{F_e}\right). \tag{34}$$

Planner and bridge passes are small and are not modeled. The script also prints the NAR-to-AR throughput ratio at compute saturation, $N/\big(K(512+N)\big)$; the batch at which AR decoding becomes compute-bound, $B^\star = (2P/\mathrm{BW}_e)/(2P/F_e)$, which is independent of model size; and the largest batch whose KV cache fits beside the weights on an 80 GB device, $(80\,\mathrm{GB} - 2P)/\mathrm{KV}(512+N)$. These outputs are the entries of Table 3 and the figures quoted in Section 5.4. They are predictions, not measurements.

B Source Code

The listing below is the complete script. Run it with `python3 pr_jepa_checks.py`.

```
"""PR-JEPA reproducibility script: numerical checks of the theory (Section 4.6) and the roofline cost model (Section 5.4)."""
import numpy as np, math
rng = np.random.default_rng(0)

print("=== (1) VICReg hinge gradient norms (per dimension, n=256) ===")
n=256; eps=1e-4
for var in [1e-1,1e-2,1e-3,1e-4,1e-6]:
    z = rng.standard_normal(n); z = (z-z.mean())/z.std(ddof=1)*math.sqrt(var)
    V = z.var(ddof=1)
    g_std = -(z-z.mean())/((n-1)*math.sqrt(V+eps))
    g_var = -2*(z-z.mean())/(n-1)
    pred_std = math.sqrt(V/((n-1)*(V+eps)))
    print(f"Var={var:.0e}  ||g_std||={np.linalg.norm(g_std):.4e} (formula {pred_std:.4e})  ||g_var||={np.linalg.norm(g_var):.4e}  ratio={np.linalg.norm(g_std)/np.linalg.norm(g_var):.2f} (1/(2sqrt(V+eps))={1/(2*math.sqrt(V+eps)):.2f})")

print("\n=== (2) Gershgorin anti-collapse bound and stable rank ===")
rng = np.random.default_rng(1)
d=32; gamma=1.0; eps=1e-4; gaps=[]; srank_ok=True
for trial in range(20000):
    diag = rng.uniform(1.0, 2.0, d)
    S = rng.standard_normal((d,d)); S=(S+S.T)/2; np.fill_diagonal(S,0); S *= rng.uniform(0,0.05)
    C = np.diag(diag)+S
    lam=np.linalg.eigvalsh(C)
    if lam.min() < 0: continue
    off=C-np.diag(np.diag(C)); c=(off**2).sum()/d
    v=np.mean(np.maximum(0,gamma-np.sqrt(np.diag(C)+eps)))
    if np.linalg.matrix_rank(C) < (np.trace(C)**2)/(np.linalg.norm(C,'fro')**2)-1e-9: srank_ok=False
    bound=gamma**2-eps-math.sqrt(d*(d-1)*c)
    if v==0 and bound>0: gaps.append(lam.min()-bound)
print(f"premise met in {len(gaps)} cases; min(lambda_min - bound) = {min(gaps):.5f} (must be >= 0)")
print("rank >= stable rank in all cases:", srank_ok)

print("\n=== (3) Euler flow-map sensitivity bound ===")
rng = np.random.default_rng(0)
dx, dz = 16, 8
maxratio = 0
for trial in range(500):
    A = rng.standard_normal((dx,dx)); A *= rng.uniform(0.2,3.0)/np.linalg.norm(A,2)
```

```
    B = rng.standard_normal((dx,dz)); B *= rng.uniform(0.2,2.0)/np.linalg.norm(B,2)
    Lx, Lz = np.linalg.norm(A,2), np.linalg.norm(B,2)
    W = rng.standard_normal((dx,dx))*0.3
    def v(x,z): return np.tanh(A@x + B@z)  # Lip_x<=Lx, Lip_z<=Lz
    for K in [1,2,4,8,16]:
        x0 = rng.standard_normal(dx); z = rng.standard_normal(dz); dzv = 1e-3*rng.standard_normal(dz)
        xa, xb = x0.copy(), x0.copy()
        for k in range(K):
            xa = xa + v(xa,z)/K; xb = xb + v(xb,z+dzv)/K
        emp = np.linalg.norm(xa-xb)/np.linalg.norm(dzv)
        bnd = (Lz/Lx)*((1+Lx/K)**K - 1)
        maxratio = max(maxratio, emp/bnd)
print(f"max over 2500 cases of empirical/bound = {maxratio:.3f} (must be <= 1)")
for K in [1,2,4,8,16]:
    Lx=1.0; print(f"  K={K}: (1+Lx/K)^K-1 = {(1+Lx/K)**K-1:.3f}  (limit e^Lx-1={math.e-1:.3f})")

print("\n=== (4) Few-step mixture-path kernel: Euler vs exact ===")
for name, kap, dkap in [("linear k=t", lambda t:t, lambda t:1.0),
                        ("sqrt k=t^0.5", lambda t:t**0.5, lambda t:0.5*t**-0.5),
                        ("cosine k=1-cos(pi t/2)", lambda t:1-math.cos(math.pi*t/2), lambda t:math.pi/2*math.
     sin(math.pi*t/2))]:
    for K in [2,4,8]:
        h=1/K; worst=0; exact_ok=True
        for k in range(K):
            t=k*h
            if t==0 and name.startswith("sqrt"): t=1e-12
            euler = h*dkap(max(t,1e-12))/(1-kap(t))
            exact = (kap(min(t+h,1))-kap(t))/(1-kap(t))
            worst=max(worst,euler); exact_ok &= (0<=exact<=1+1e-12)
        print(f"  {name:24s} K={K}: max Euler jump mass h*kdot/(1-k) = {worst:8.3f}   exact in [0,1]: {exact_ok}
     ")

print("\n=== (5) Harmonic sum of jump masses (linear schedule) ===")
for K in [1,2,4,8,16]:
    print(f"  K={K}: sum_k pi_k = H_K = {sum(1/(K-k) for k in range(K)):.3f}")

print("\n=== (6) Risk-calibrated gate ===")
for Hmax, tau, alpha in [(16,1.0,0.05),(32,1.0,0.05),(64,1.0,0.1)]:
    beta = 2*math.sqrt(Hmax)/(math.e*tau*math.sqrt(alpha))
    sig = np.linspace(1e-4,5,200000)
    risk = Hmax*np.exp(-beta*sig)*sig**2/tau**2
    print(f"  Hmax={Hmax}, tau={tau}, alpha={alpha}: beta_min={beta:.3f}; max risk over sigma = {risk.max():.4f}
      (<= alpha)")

print("\n=== (7) Logit-KL posterior velocity identity check ===")
V=7; epsl=0.05; t=0.6
a = math.log((1-epsl+epsl/V)/(epsl/V)); b = math.log(epsl/V)
p = rng.dirichlet(np.ones(V))
l0 = rng.standard_normal(V)
# sample many (k, l0) consistent with a fixed l_t is hard; check algebra instead: v = l1 - l0 with l0=(lt - t
     l1)/(1-t)
k = 3; l1 = b*np.ones(V); l1[k]+=a
lt = (1-t)*l0 + t*l1
v_true = l1 - l0
v_form = (l1 - lt)/(1-t)
print("  max|(l1-l0) - (l1-lt)/(1-t)| =", np.abs(v_true-v_form).max())


# ===== Roofline cost model (Section 5.4) =====
# Hardware (vendor peak specs): H100 SXM
BW = 3.35e12      # HBM bytes/s
F  = 989e12       # dense bf16 FLOP/s
eta_bw, eta_f = 0.7, 0.5   # assumed achieved fractions
BWe, Fe = BW*eta_bw, F*eta_f
bytes_per = 2
# Model configs (public configs)
models = {
 "1.3B (FS-DFM scale)": dict(P=1.3e9, L=24, kvh=16, hd=128, d=2048),   # MHA-style assumption
 "7.6B (Qwen2.5-Coder-7B)": dict(P=7.6e9, L=28, kvh=4, hd=128, d=3584), # GQA
}
prompt = 512
def kv_bytes_per_tok(m): return 2*m["L"]*m["kvh"]*m["hd"]*bytes_per
def ar_latency(m, N):
    W = m["P"]*bytes_per
    t = 0.0
    # prefill (compute bound)
    t += 2*m["P"]*prompt/Fe
    for i in range(N):
        ctx = prompt+i
        mem = W + kv_bytes_per_tok(m)*ctx
        flops = 2*m["P"] + 4*m["L"]*ctx*m["d"]
```

```
        t += max(mem/BWe, flops/Fe)
    return t
def nar_latency(m, N, K, planner_ms=0.0):
    T = prompt+N
    flops = 2*m["P"]*T + 4*m["L"]*T*T*m["d"]
    W = m["P"]*bytes_per
    step = max(W/BWe, flops/Fe)
    return K*step + planner_ms/1e3
print("KV cache per token (KiB):", {k: round(kv_bytes_per_tok(m)/1024,1) for k,m in models.items()})
print()
print("| Model | N | AR latency (s) | NAR K=4 (s) | NAR K=8 (s) | AR/NAR(K=8) | AR KV cache/seq (MiB) |")
for name,m in models.items():
    for N in [256,1024,4096]:
        a=ar_latency(m,N); n4=nar_latency(m,N,4); n8=nar_latency(m,N,8)
        kv = kv_bytes_per_tok(m)*(prompt+N)/2**20
        print(f"| {name} | {N} | {a:.2f} | {n4:.3f} | {n8:.3f} | {a/n8:.1f}x | {kv:.0f} |")
print()
# throughput at saturation
print("Throughput ratio at compute saturation NAR/AR ~ N/(K*(prompt+N)):")
for N in [256,1024,4096]:
    for K in [4,8]:
        print(f"  N={N} K={K}: {N/(K*(prompt+N)):.3f}")
# Crossover batch where AR decode becomes compute-bound (ignoring KV reads)
for name,m in models.items():
    Bstar = (m["P"]*bytes_per/BWe)/(2*m["P"]/Fe)
    free = 80e9 - m["P"]*bytes_per
    for N in [1024,8192]:
        Bmem = free/(kv_bytes_per_tok(m)*(prompt+N))
        print(f"{name}: B* (compute-bound crossover) ~ {Bstar:.0f}; KV-limited max batch at N={N}: {Bmem:.0f}")
```